

Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Kierunek: Informatyka
Specjalność: Inżynieria Systemów Informatycznych

PRACA DYPLOMOWA
MAGISTERSKA

**Projekt i analiza porównawcza wydajności skalowalnych architektur
systemów typu SaaS**

Oskar Siemieniuk

Opiekun pracy
dr inż. Konrad Kluwak

Słowa kluczowe: skalowalność, Kubernetes, HPA, mikroserwisy, SaaS, testy obciążeniowe,
Prometheus, autoskalowanie

Spis treści

1	Wstęp teoretyczny	1
1.1	Motywacja i problem badawczy	1
1.2	Hipoteza badawcza, cel i zakres	2
1.2.1	Hipoteza	2
1.2.2	Cel i zadania	2
1.2.3	Zakres	2
1.3	Podstawy teoretyczne	2
1.3.1	Architektury skalowalne: od monolitu do mikroserwisów	2
1.3.2	Wzorce skalowania i autoskalowania	3
1.3.3	Kubernetes i orkiestracja kontenerów	3
1.3.4	Mechanizmy autoskalowania w Kubernetes	3
1.3.5	Model SaaS i multi-tenancy	4
1.4	Przegląd literatury	4
1.4.1	Istniejące badania nad HPA	4
1.4.2	Zidentyfikowane luki badawcze	5
1.5	Struktura pracy	5
2	Przegląd technologii	7
2.1	Google Kubernetes Engine — zarządzany Kubernetes w GCP	7
2.2	GCP LoadBalancer — ekspozycja serwisów	7
2.3	Compute Engine — generator obciążenia	8
2.4	kube-prometheus-stack — stos monitorujący	8
2.4.1	Prometheus	9
2.4.2	Grafana	9
2.4.3	Prometheus Adapter	9
2.5	k6 — narzędzie do testów obciążeniowych	11
2.6	Docker i konteneryzacja	11
2.7	Podsumowanie stosu technologicznego	12
3	Projekt architektury systemu badawczego	13
3.1	Macierz badawcza	13
3.2	Aplikacje badawcze	13
3.2.1	CPU-service — obciążenie CPU-bound	14
3.2.2	IO-service — obciążenie I/O-bound	14
3.2.3	Echo-server — serwis pomocniczy	14
3.2.4	Memory-service — serwis pamięciowy	14
3.3	Strategie HPA	14
3.4	Topologia wdrożeniowa	15
3.5	Infrastruktura monitorująca i eksport danych	16

3.6	Plan eksperymentów	16
3.6.1	Scenariusze testowe	16
3.6.2	Procedura	16
3.6.3	Profile obciążeniowe k6	16
3.6.4	Metryki badawcze	17
3.6.5	Analiza zjawisk przejściowych	17
3.7	Podsumowanie decyzji projektowych	17
4	Implementacja	19
4.1	Struktura repozytorium	19
4.2	Implementacja mikroservisów	19
4.2.1	CPU-service	20
4.2.2	IO-service	20
4.2.3	Echo-server i Memory-service	22
4.3	Konfiguracja Kubernetes	22
4.3.1	Deploymenty	22
4.3.2	HPA — trzy warianty	23
4.3.3	Services	24
4.4	Infrastruktura monitorująca	25
4.4.1	kube-prometheus-stack	25
4.4.2	Prometheus Adapter	25
4.4.3	Grafana — dashboard i eksport	26
4.5	Skrypty automatyzujące	26
4.5.1	run-tests.sh — orkiestracja eksperymentów	26
4.6	Scenariusze testowe k6	28
4.7	Podsumowanie implementacji	29
5	Badania eksperymentalne i analiza wyników	31
5.1	Założenia redakcyjne rozdziału (anti-generic)	31
5.2	Metodyka i zakres danych	31
5.3	Wyniki ilościowe	32
5.3.1	Scenariusze bazowe	32
5.3.2	Scenariusze CPU-bound	32
5.3.3	Scenariusze I/O-bound	32
5.4	Wykresy przebiegu skalowania i zasobów	33
5.5	Interpretacja wyników	37
5.5.1	Dlaczego HPA CPU działa w obu profilach?	37
5.5.2	Dlaczego HPA Memory nie skaluje?	37
5.5.3	Dlaczego HPA Custom RPS nie skaluje?	37
5.6	Weryfikacja hipotezy i wniosek roboczy	37
6	Podsumowanie	39
6.1	Podsumowanie zrealizowanych prac	39
6.2	Weryfikacja hipotezy badawczej	40
6.3	Ograniczenia pracy	40
6.4	Rekomendacje praktyczne	41
6.5	Dalsze kierunki badań	41
6.6	Podsumowanie końcowe	42

Rozdział 1

Wstęp teoretyczny

1.1 Motywacja i problem badawczy

Systemy informatyczne dostarczane w modelu *Software as a Service* (SaaS) stoją przed podwójnym wyzwaniem: muszą jednocześnie obsługiwać zmienne obciążenie generowane przez wielu klientów (tenantów) oraz utrzymywać przewidywalną wydajność przy racjonalnym koszcie infrastruktury. Współdzielenie zasobów obliczeniowych przez tenantów o różnych profilach aktywności — od lekkiego przeglądania dashboardu po masowy eksport danych — oznacza, że system nie może polegać na statycznym przydziale zasobów. Potrzebuje mechanizmu, który dynamicznie dostosuje liczbę jednostek obliczeniowych do bieżącego zapotrzebowania.

W ekosystemie Kubernetes podstawowym narzędziem realizującym ten cel jest *HorizontalPodAutoscaler* (HPA) — kontroler, który na podstawie obserwowanych metryk podejmuje decyzje o dodaniu lub usunięciu replik aplikacji. HPA działa reaktywnie: co 15 sekund porównuje bieżącą wartość wybranej metryki z zadanym progiem i na tej podstawie oblicza pożądaną liczbę replik. Prostota tego mechanizmu jest jego zaletą operacyjną, ale jednocześnie rodzi pytanie: **czy wybór metryki ma znaczenie dla skuteczności skalowania?**

Intuicja inżynierska podpowiada, że dla aplikacji obliczeniowej (CPU-bound) naturalnym wyborem będzie HPA oparty na zużyciu procesora, a dla aplikacji oczekującej na zewnętrzne odpowiedzi (I/O-bound) — HPA oparty na liczbie żądań. Problem w tym, że w praktyce zespoły wdrożeniowe rzadko analizują profil obciążeniowy swojej aplikacji przed skonfigurowaniem autoskalowania. Domyślna konfiguracja HPA — metryka CPU z progiem 50% — jest stosowana uniwersalnie, niezależnie od charakterystyki serwisu. W środowisku SaaS, gdzie jeden klaster obsługuje dziesiątki tenantów, konsekwencje błędnego doboru metryki mogą być poważne: od niedoskalowania (spadek wydajności dla wszystkich tenantów) po przeskalowanie (niepotrzebny koszt infrastruktury chmurowej).

Niniejsza praca podejmuje próbę systematycznego rozstrzygnięcia tej kwestii poprzez kontrolowane eksperymenty w środowisku Google Kubernetes Engine (GKE). Zamiast zakładać z góry, która metryka jest najlepsza, badanie traktuje wybór metryki HPA jako zmienną niezależną i mierzy jej wpływ na kluczowe wskaźniki wydajnościowe: przepustowość, opóźnienia, czas reakcji na skalowanie oraz stopę błędów.

1.2 Hipoteza badawcza, cel i zakres

1.2.1 Hipoteza

Hipoteza badawcza pracy brzmi:

Nie istnieje jedna uniwersalna metryka HPA — skuteczność strategii skalowania zależy od charakterystyki obciążeniowej aplikacji (workload profile). Błędny wybór metryki prowadzi do niedoskalowania (underscaling) lub przeskalowania (overscaling).

Przez „skuteczność” rozumie się zdolność HPA do wykrycia przeciążenia i dodania replik zanim opóźnienia odpowiedzi przekroczą akceptowalny próg, przy jednoczesnym unikaniu niepotrzebnego skalowania w górę gdy obciążenie jest niskie.

1.2.2 Cel i zadania

Celem pracy jest zaprojektowanie, implementacja oraz eksperymentalna walidacja prototypu skalowalnego systemu w modelu SaaS, ze szczególnym uwzględnieniem porównania trzech strategii HPA dla dwóch diametralnie różnych profili obciążeniowych. Cel ten został zrealizowany poprzez następujące zadania:

1. Zaprojektowanie i implementacja dwóch mikroservisów o skrajnie odmiennych charakterystykach: CPU-bound (przetwarzanie obrazu, obliczenia rekurencyjne) oraz I/O-bound (łańcuchy asynchronicznych opóźnień symulujące zapytania do zewnętrznych usług).
2. Przeprowadzenie kontrolowanych eksperymentów porównujących trzy strategie HPA: opartą na zużyciu CPU, opartą na zużyciu pamięci oraz opartą na niestandardowej metryce RPS (Requests Per Second), dostarczanej przez prometheus-adapter.
3. Analiza porównawcza efektywności strategii skalowania w macierzy badawczej 2×3 (dwa typy obciążenia $\times$ trzy strategie HPA).
4. Sformułowanie rekomendacji praktycznych dotyczących doboru metryki HPA w zależności od charakterystyki aplikacji.

1.2.3 Zakres

Zakres pracy obejmuje analizę teoretyczną zagadnień związanych z systemami rozproszonymi i autoskalowaniem, budowę autorskiego rozwiązania systemowego w technologii Kubernetes na platformie Google Cloud, przygotowanie środowiska testowego z pełnym stosem monitorującym (Prometheus, Grafana, k6) oraz przeprowadzenie serii badań eksperymentalnych weryfikujących efektywność zastosowanych technik skalowania.

1.3 Podstawy teoretyczne

1.3.1 Architektury skalowalne: od monolitu do mikroservisów

Ewolucja architektur systemów rozproszonych — od monolitycznych aplikacji, przez architekturę zorientowaną na usługi (SOA), do współczesnego paradygmatu mikroservisów

— była w dużej mierze napędzana potrzebą elastycznego skalowania. W monolicie cała funkcjonalność systemu jest zawarta w jednej jednostce wdrożeniowej; skalowanie odbywa się przez uruchomienie wielu identycznych instancji całej aplikacji, co jest nieefektywne, gdy tylko wybrane komponenty wymagają dodatkowej mocy obliczeniowej. SOA wprowadziło podział na usługi komunikujące się przez Enterprise Service Bus, umożliwiając selektywne skalowanie kosztem złożoności warstwy integracyjnej. Mikroserwisy wynoszą tę ideę na kolejny poziom: każda funkcja biznesowa jest niezależną jednostką wdrożeniową z własnym cyklem życia, stosem technologicznym i — co kluczowe z perspektywy tej pracy — własną polityką skalowania. To właśnie w architekturze mikroserwisowej precyzyjny dobór metryki HPA staje się decyzją o wymiernych konsekwencjach wydajnościowych i kosztowych.

1.3.2 Wzorce skalowania i autoskalowania

Skalowanie systemów realizuje się w trzech podstawowych wymiarach. Skalowanie pionowe (vertical scaling) polega na zwiększaniu zasobów pojedynczej instancji — więcej CPU, więcej pamięci RAM. Jest ograniczone fizycznymi możliwościami sprzętu i nie zwiększa odporności na awarie. Skalowanie poziome (horizontal scaling) zwiększa liczbę instancji obsługujących ruch, co rozkłada obciążenie i podnosi niezawodność, ale wymaga mechanizmów równoważenia obciążenia oraz koordynacji stanu. Trzeci wymiar — skalowanie automatyczne (autoscaling) — to dynamiczne dostosowywanie liczby instancji na podstawie obserwowanych metryk, bez interwencji operatora. W systemach chmurowych, gdzie koszty są proporcjonalne do wykorzystania zasobów, autoskalowanie jest nie tyle optymalizacją, co warunkiem koniecznym ekonomicznej eksploatacji.

1.3.3 Kubernetes i orkiestracja kontenerów

Kubernetes (K8s) jest otwartą platformą do automatyzacji wdrażania, skalowania i zarządzania aplikacjami kontenerowymi, utrzymywaną przez Cloud Native Computing Foundation (CNCF). Architektura klastra Kubernetes dzieli się na dwie warstwy. Control Plane — zarządzana przez Google w przypadku GKE — obejmuje API server, rozproszony magazyn stanu etcd, scheduler przydzielający Pody do węzłów oraz controller manager, w którym działa m.in. HorizontalPodAutoscaler. Worker Nodes to węzły wykonawcze, na których uruchamiane są Pody — podstawowe jednostki obliczeniowe, czyli grupy jednego lub więcej kontenerów współdzielących przestrzeń sieciową i woluminy. Pody są efemeryczne: mogą być tworzone, niszczone i przenoszone między węzłami, co czyni je naturalnym celem dla mechanizmów autoskalowania.

1.3.4 Mechanizmy autoskalowania w Kubernetes

HorizontalPodAutoscaler (HPA) jest kontrolerem, który automatycznie dostosowuje liczbę replik w Deployment, ReplicaSet lub StatefulSet na podstawie obserwowanych metryk. Algorytm HPA działa według formuły:

$$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \times \frac{\text{currentMetricValue}}{\text{desiredMetricValue}} \right\rceil \quad (1.1)$$

HPA w wersji API `autoscaling/v2` — wykorzystywanej w niniejszej pracy — obsługuje trzy typy metryk: resource metrics (metryki zasobów kontenera, takie jak CPU

i pamięć, dostarczane przez metrics-server), pods metrics (metryki niestandardowe skojarzone z konkretnymi Podami, dostarczane przez zewnętrzne adaptery, np. prometheus-adapter) oraz object metrics (metryki opisujące obiekty Kubernetes, również dostarczane przez adaptery).

Dodatkowo, HPA implementuje mechanizmy stabilizacyjne zapobiegające zbyt częstym zmianom liczby replik. Okno stabilizacji skalowania w dół (scale-down stabilization window) wynosi domyślnie 5 minut — HPA uwzględnia maksymalną wartość replik z tego okna przed podjęciem decyzji o redukcji. Tolerancja (10%) oznacza, że metryka musi odchylić się o więcej niż 10% od progu, aby wywołać skalowanie. Od wersji `autoscaling/v2` można również definiować polityki ograniczające tempo skalowania, np. maksymalnie 2 nowe repliki na 60 sekund.

VerticalPodAutoscaler (VPA) dostosowuje resource requests i limits dla kontenerów w Podzie, zmieniając ilość zasobów przydzielonych każdej replice zamiast liczby replik. **KEDA** (Kubernetes Event-Driven Autoscaling) rozszerza HPA o skalowanie na podstawie zdarzeń z zewnętrznych systemów (np. długość kolejki Apache Kafka) oraz o możliwość skalowania do zera replik. VPA i KEDA nie są przedmiotem badań w niniejszej pracy, ale stanowią istotne kierunki dalszych prac.

1.3.5 Model SaaS i multi-tenancy

Model Software as a Service jest jedną z trzech podstawowych kategorii cloud computing (obok IaaS i PaaS), w której dostawca udostępnia użytkownikom gotową aplikację przez Internet, zarządzając całą infrastrukturą. Kluczową cechą SaaS jest wielodostępność (multi-tenancy) — architektoniczny wzorzec umożliwiający obsługę wielu klientów przez pojedynczą instancję systemu, przy zachowaniu logicznej izolacji danych i przewidywalnej wydajności dla każdego tenanta.

W kontekście Kubernetes można wyróżnić kilka poziomów izolacji: od najbliższego (application-level multi-tenancy, gdzie tenanci są rozróżniani przez nagłówek HTTP i współdzielą ten sam namespace), przez namespace-per-tenant (każdy tenant otrzymuje własny namespace z izolowanymi zasobami), aż po cluster-per-tenant (dedykowany klaster dla każdego tenanta, zapewniający najwyższy poziom izolacji przy najwyższym koszcie). Współdzielenie infrastruktury oznacza, że decyzje HPA podejmowane w odpowiedzi na obciążenie jednego tenanta mogą wpływać na wydajność pozostałych — jest to jedno z głównych pytań motywujących niniejsze badanie.

1.4 Przegląd literatury

1.4.1 Istniejące badania nad HPA

Problematyka autoskalowania w Kubernetes była przedmiotem licznych badań w ostatnich latach. Casalicchio i Perciballi [1] przeprowadzili analizę porównawczą domyślnego HPA z KEDA, badając czas reakcji i wykorzystanie zasobów dla obciążeń o różnych profilach. Autorzy wykazali, że KEDA oferuje szybszy czas skalowania przy obciążeniach zdarzeniowych, ale domyślny HPA jest wystarczający dla obciążeń o stałym charakterze — nie analizowali jednak wpływu konkretnego wyboru metryki w obrębie samego HPA.

Nguyen i Kim [2] zaproponowali predykcyjny algorytm skalowania oparty na sieciach LSTM (Long Short-Term Memory), który przewiduje przyszłe obciążenie i skaluje proaktywnie, redukując liczbę naruszeń SLA o 30% w porównaniu do reaktywnego HPA.

Podejście to wymaga jednak znacznych zasobów obliczeniowych na trenowanie modelu i nie zostało zweryfikowane w środowisku wielodostępnym (SaaS).

1.4.2 Zidentyfikowane luki badawcze

Na podstawie przeglądu literatury zidentyfikowano następujące luki badawcze, które motywują niniejszą pracę:

1. **Brak systematycznego porównania strategii HPA dla różnych profili obciążeniowych** — większość badań testuje HPA na jednorodnym obciążeniu (wyłącznie CPU-bound lub wyłącznie I/O-bound), nie analizując interakcji między profilem obciążeniowym a wyborem metryki.
2. **Ograniczona analiza HPA Memory** — strategia oparta na zużyciu pamięci jest rzadko badana empirycznie, ponieważ zakłada się z góry, że pamięć w większości aplikacji nie koreluje z obciążeniem. Niniejsza praca dostarcza danych eksperymentalnych weryfikujących to założenie.
3. **Brak badań łączących HPA z modelem SaaS** — istniejące badania nad multi-tenancy koncentrują się na izolacji danych i bezpieczeństwie, pomijając wpływ wielodostępności na efektywność autoskalowania.
4. **Brak systematycznej analizy zjawisk przejściowych** — istniejące publikacje koncentrują się na metrykach zagregowanych (średni RPS, średnie opóźnienie), pomijając analizę zjawisk takich jak opóźnienie reakcji HPA (HPA Lag), opóźnienie zimnego startu Podów (Cold Start Latency) czy kaskadowe błędy przy CPU throttlingu. Niniejsza praca identyfikuje i dokumentuje te zjawiska na podstawie danych eksperymentalnych.

1.5 Struktura pracy

Praca składa się z sześciu rozdziałów. Rozdział 2 zawiera szczegółowy przegląd technologii wykorzystanych w projekcie, ze szczególnym uwzględnieniem komponentów, które nie zostały omówione w części teoretycznej: Google Kubernetes Engine jako zarządzanej platformy orkiestracji, stosu monitorującego kube-prometheus-stack, narzędzia do testów obciążeniowych k6 oraz prometheus-adapter jako mostka między metrykami aplikacyjnymi a API HPA. Rozdział 3 przedstawia projekt architektury systemu badawczego — macierz badawczą 2×3 , specyfikację mikroservisów, trzy warianty HPA oraz plan eksperymentów. Rozdział 4 dokumentuje implementację: strukturę repozytorium, kod mikroservisów, konfigurację Kubernetes i infrastrukturę monitorującą. Rozdział 5 zawiera wyniki badań eksperymentalnych przeprowadzonych na klastrze GKE wraz z ich analizą i interpretacją. Rozdział 6 zamyka pracę podsumowaniem, weryfikacją hipotezy, rekomendacjami praktycznymi oraz wskazaniem ograniczeń i dalszych kierunków badań.

Rozdział 2

Przegląd technologii

W rozdziale tym omówiono technologie wykorzystane w projekcie badawczym, które nie zostały szczegółowo opisane w części teoretycznej (Rozdział 1). Nacisk położono na komponenty specyficzne dla środowiska Google Cloud Platform (GCP): zarządzany klastrer GKE, równoważenie obciążenia przez GCP LoadBalancer, maszynę Compute Engine jako generator obciążenia testowego oraz stos monitorujący kube-prometheus-stack z prometheus-adapter jako mostkiem między metrykami aplikacyjnymi a API HPA.

2.1 Google Kubernetes Engine — zarządzany Kubernetes w GCP

Google Kubernetes Engine (GKE) jest zarządzaną usługą Kubernetes oferowaną przez Google Cloud Platform, certyfikowaną przez CNCF pod kątem pełnej zgodności API. W odróżnieniu od samodzielnie administrowanego klastra, GKE automatycznie zarządza warstwą Control Plane — użytkownik nie ma bezpośredniego dostępu do komponentów takich jak etcd czy kube-apiserver, co eliminuje konieczność ich ręcznej konfiguracji i utrzymania.

W projekcie wykorzystano klastrer GKE w konfiguracji VPC-native na maszynach typu `e2-standard-4` (4 vCPU, 16 GB RAM). Wybór GKE jako platformy badawczej podyktowany był czterema czynnikami. Po pierwsze, pełna zgodność API gwarantuje, że wszystkie manifesty (Deployment, HPA, Service) działają identycznie jak na dowolnym klastrze zgodnym z CNCF. Po drugie, integracja z GCP LoadBalancer umożliwia bezpośrednią ekspozycję serwisów aplikacyjnych na zewnętrzne adresy IP bez potrzeby tunelowania. Po trzecie, maszyna generująca obciążenie testowe (Compute Engine) znajduje się w tej samej sieci VPC co klastrer, co zapewnia niskie opóźnienie sieciowe ($\sim 1\text{--}5$ ms). Po czwarte, GKE cluster autoscaler automatycznie dostosowuje liczbę węzłów do zapotrzebowania, umożliwiając testy z dużą liczbą Podów bez ręcznej interwencji.

2.2 GCP LoadBalancer — ekspozycja serwisów

W środowisku GKE serwisy typu LoadBalancer są automatycznie integrowane z infrastrukturą równoważenia obciążenia GCP. Po utworzeniu obiektu Service typu LoadBalancer, kontroler GKE (cloud controller manager) tworzy regułę forwarding rule w GCP, a platforma przydziela publiczny adres IP (regionalny, statyczny lub efemeryczny). Ruch przychodzący na ten adres jest dystrybuowany do Podów przez kube-proxy na węzłach.

W projekcie wszystkie serwisy aplikacyjne są eksponowane przez LoadBalancer, co umożliwia maszynie generującej obciążenie testowe bezpośrednie wysyłanie żądań HTTP. Rozwiązanie to jest istotnie lepsze od powszechnie stosowanego w środowiskach deweloperskich mechanizmu `kubectl port-forward`, który tuneluje ruch przez pojedyncze połączenie WebSocket do API server — przy dużym obciążeniu tworzy to wąskie gardło i dodaje nienaturalne opóźnienia. GCP LoadBalancer eliminuje te ograniczenia, zapewniając bezpośrednią ścieżkę sieciową między generatorem obciążenia a Podami, co odzwierciedla rzeczywiste warunki produkcyjne.

2.3 Compute Engine — generator obciążenia

Google Compute Engine (GCE) to usługa maszyn wirtualnych GCP. W projekcie wykorzystano dedykowaną maszynę (nazwaną `k6-generator`) jako wyłącznego hosta dla narzędzia `k6`, generującego obciążenie testowe. Umieszczenie generatora na osobnej maszynie w tej samej sieci VPC co klaster GKE rozwiązuje dwa problemy: zapobiega zakłócaniu wyników testów przez konkurencję o zasoby (generator i badane serwisy nie dzielą CPU ani pamięci) oraz zapewnia realistyczne, niskie opóźnienia sieciowe między nadawcą a odbiorcą ruchu.

Zamiast konfiguracji DNS (która wymagałaby rejestracji domen i propagacji rekordów), skrypty `k6` wykorzystują mechanizm `options.hosts` do bezpośredniego mapowania nazw serwisów na adresy IP GCP LoadBalancer. Mechanizm ten działa na poziomie biblioteki HTTP `k6` i podmienia nagłówki `Host` oraz docelowy adres IP dla każdego żądania.

```
1 export const options = {  
2   hosts: {  
3     'cpu-service': '34.118.71.122',  
4     'io-service': '35.204.93.45',  
5   },  
6   scenarios: { ... }  
7 };
```

Listing 2.1 Mapowanie hostów w opcjach skryptu `k6`

2.4 kube-prometheus-stack — stos monitorujący

`kube-prometheus-stack` jest pakietem Helm (Helm chart) rozwijanym przez społeczność `prometheus-community`, który instaluje kompletny stos monitorujący dla Kubernetes. W skład pakietu wchodzi: Prometheus (system zbierania i przechowywania metryk w postaci szeregów czasowych), Grafana (platforma wizualizacji danych), `kube-state-metrics` (komponent generujący metryki o stanie obiektów Kubernetes), `node-exporter` (komponent zbierający metryki systemowe z węzłów) oraz Prometheus Operator (kontroler zarządzający cyklem życia instancji Prometheusa).

Wybór instalacji przez Helm — zamiast ręcznego wdrażania pojedynczych manifestów YAML — jest podyktowany automatyczną konfiguracją: `ServiceMonitor` i `PodMonitor` automatycznie wykrywają źródła metryk na podstawie etykiet, bez konieczności ręcznej edycji pliku `prometheus.yml`. Ponadto, Grafana otrzymuje zestaw gotowych dashboardów dla Kubernetes, a Prometheus Operator zarządza aktualizacjami i skalowaniem.

Pakiet instaluje się jednym zestawem poleceń:

```
1 helm repo add prometheus-community \
2   https://prometheus-community.github.io/helm-charts
3 helm repo update
4 helm install monitoring prometheus-community/kube-prometheus-stack \
5   --namespace magisterka --create-namespace \
6   --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=
   false \
7   --set grafana.service.type=LoadBalancer
```

Listing 2.2 Instalacja kube-prometheus-stack przez Helm

Parametr `grafana.service.type=LoadBalancer` eksponuje Grafanę publicznie przez GCP LoadBalancer, umożliwiając obserwację eksperymentów w czasie rzeczywistym z dowolnej przeglądarki.

2.4.1 Prometheus

Prometheus jest otwartoźródłowym systemem monitorowania, pierwotnie opracowanym w SoundCloud, obecnie inkubowanym przez CNCF. Jego architektura opiera się na modelu pull: Prometheus aktywnie pobiera (scrape) metryki z aplikacji poprzez HTTP w konfigurowalnych interwałach — w projekcie co 15 sekund. Zebrane dane są przechowywane w natywnej bazie szeregów czasowych (TSDB) z etykietami umożliwiającymi filtrowanie i agregację. Język zapytań PromQL pozwala na zaawansowane operacje: agregacje, operacje na zakresach czasowych oraz predykcję.

W projekcie Prometheus pełni podwójną rolę. Zbiera metryki infrastrukturalne (CPU, RAM, liczba replik, stan HPA) poprzez kube-state-metrics i node-exporter oraz metryki aplikacyjne z mikroserwisów (RPS, czas przetwarzania, liczba aktywnych zapytań) z endpointów `/metrics`. Jednocześnie udostępnia te dane przez API dla prometheus-adapter, który transformuje zapytania PromQL na format zrozumiały dla HPA Kubernetes.

2.4.2 Grafana

Grafana jest platformą do wizualizacji danych, integrującą się z wieloma źródłami, w tym z Prometheusem. W projekcie wykorzystano autorski dashboard „Magisterka – System Overview” zawierający 16 paneli odświeżanych co 10 sekund, pokazujących kluczowe metryki: RPS, zużycie CPU i pamięci per Pod, percentyle opóźnień (p50, p95, p99), liczbę replik oraz stan HPA. Dashboard jest dostępny publicznie przez GCP LoadBalancer.

Dla każdego scenariusza testowego z Grafany eksportowanych jest 5 zestawów danych CSV: liczba replik w czasie, zużycie CPU per Pod (mCPU), zużycie pamięci per Pod (MiB), przepustowość (RPS) oraz percentyle opóźnień (ms). Dane te, wraz z wynikami k6, stanowią kompletny zbiór danych badawczych dla analizy w Rozdziale 5.

2.4.3 Prometheus Adapter

Prometheus Adapter (wersja v0.12.0) jest komponentem implementującym Kubernetes Custom Metrics API. Jego rola polega na rejestracji jako APIService w Kubernetes (rozszerzając API o ścieżki `/apis/custom.metrics.k8s.io/`), pobieraniu metryk z Prometheusa poprzez zapytania PromQL oraz serwowaniu tych metryk w formacie oczekiwanym przez HPA.

Konfiguracja adaptera jest przechowywana w ConfigMap (`prometheus-adapter-config`) i definiuje reguły mapowania metryk Prometheusa na metryki Kubernetes. W projekcie

zdefiniowano cztery reguły: dwie dla metryki RPS (przeliczanej przez funkcję `rate()` [1m] z liczników żądań HTTP) oraz dwie dla metryki liczby żądań w toku (in-flight) — po jednej parze dla `cpu-service` i `io-service`.

```

1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: prometheus-adapter-config
5    namespace: magisterka
6    labels:
7      app: prometheus-adapter
8      tier: monitoring
9  data:
10   config.yaml: |
11     rules:
12       - seriesQuery: 'cpu_service_http_requests_total{namespace!="",pod
13         !=""}'
14       resources:
15         overrides:
16           namespace: {resource: "namespace"}
17           pod: {resource: "pod"}
18       name:
19         matches: "cpu_service_http_requests_total"
20         as: "http_requests_per_second"
21       metricsQuery: 'sum(rate(cpu_service_http_requests_total{<<.
22         LabelMatchers>>}[1m])) by (<<.GroupBy>>)'
23       - seriesQuery: 'cpu_service_http_requests_inprogress{namespace!="",
24         pod!=""}'
25       resources:
26         overrides:
27           namespace: {resource: "namespace"}
28           pod: {resource: "pod"}
29       name:
30         matches: "cpu_service_http_requests_inprogress"
31         as: "http_requests_inflight"
32       metricsQuery: 'sum(cpu_service_http_requests_inprogress{<<.
33         LabelMatchers>>}) by (<<.GroupBy>>)'
34       - seriesQuery: 'io_service_http_requests_total{namespace!="",pod!=""}
35         ,
36       resources:
37         overrides:
38           namespace: {resource: "namespace"}
39           pod: {resource: "pod"}
40       name:
41         matches: "io_service_http_requests_total"
42         as: "http_requests_per_second"
43       metricsQuery: 'sum(rate(io_service_http_requests_total{<<.
44         LabelMatchers>>}[1m])) by (<<.GroupBy>>)'
45       - seriesQuery: 'io_service_http_requests_inprogress{namespace!="",pod
46         !=""}'
47       resources:
48         overrides:
49           namespace: {resource: "namespace"}
50           pod: {resource: "pod"}
51       name:
52         matches: "io_service_http_requests_inprogress"
53         as: "http_requests_inflight"
54       metricsQuery: 'sum(io_service_http_requests_inprogress{<<.
55         LabelMatchers>>}) by (<<.GroupBy>>)'

```

```
48 - seriesQuery: '{__name__=~"^container_.*",namespace!="",pod!=""}'
49   resources:
50     overrides:
51       namespace: {resource: "namespace"}
52       pod: {resource: "pod"}
53   name:
54     matches: "^container_(.*)$"
55     as: ""
56   metricsQuery: 'sum(<<.Series>>{<<.LabelMatchers>>,container!="POD",
    container!=""}) by (<<.GroupBy>>)'
```

Listing 2.3 Konfiguracja Prometheus Adapter — reguły mapowania metryk

2.5 k6 — narzędzie do testów obciążeniowych

k6 (Grafana k6) jest otwartoźródłowym narzędziem do testów obciążeniowych, napisanym w Go z interfejsem skryptowym w JavaScript (ES6). Pojedyncza instancja k6 jest w stanie generować setki wirtualnych użytkowników (VU, Virtual Users) bez potrzeby architektury rozproszonej, co czyni ją odpowiednią do testów średniej skali bez dodatkowej infrastruktury.

W projekcie zdefiniowano trzy skrypty k6 odpowiadające profilom obciążeniowym i scenariuszom badawczym. Skrypt `cpu-load-test.js` zawiera dwa scenariusze: Fibonacci (ramp-up od 1 do 50 VU przez 60 sekund, następnie stałe obciążenie 50 VU przez 120 sekund) oraz przetwarzanie obrazu (stałe 10 VU wysyłających losowo generowane obrazy PNG do endpointu `POST /process`). Skrypt `io-load-test.js` zawiera trzy scenariusze: zapytania ze stałym opóźnieniem (ramp-up 1→100 VU), test obciążeniowy z rosnącym opóźnieniem (50→2000 ms) oraz wywołania zewnętrzne. Skrypt `combined-scenario.js` uruchamia trzy scenariusze równolegle: Fibonacci (30 VU), przetwarzanie obrazu (5 VU) oraz zapytania IO (80 VU).

Każdy skrypt definiuje progi akceptacji (thresholds): percentyl p95 czasu odpowiedzi poniżej 5000 ms dla scenariuszy CPU oraz poniżej 10000 ms dla scenariuszy IO, a także dopuszczalną stopę błędów HTTP (< 1% dla CPU, < 5% dla IO). Przekroczenie tych progów oznacza niepowodzenie testu.

2.6 Docker i konteneryzacja

Każdy mikroservis w projekcie jest konteneryzowany za pomocą Docker. Wszystkie Dockerfile opierają się na wspólnym wzorcu: obraz bazowy `python:3.12-slim` (minimalny rozmiar), instalacja zależności przez `pip install --no-cache-dir`, aplikacja jako pojedynczy plik `main.py`, uruchomienie przez Uvicorn z parametrami `--host 0.0.0.0 --port 8000`.

Obrazy są budowane lokalnie i przesyłane do GCP Artifact Registry, skąd GKE pobiera je automatycznie podczas wdrażania na podstawie pola `image` w Deployment. W projekcie nie wykorzystuje się Docker Compose do orkiestracji testów — wszystkie eksperymenty są przeprowadzane bezpośrednio na klastrze GKE, co zapewnia zgodność środowiska testowego z produkcyjnym. Plik `docker-compose.yaml` znajdujący się w repozytorium służy wyłącznie do opcjonalnego, szybkiego uruchomienia serwisów w środowisku deweloperskim.

2.7 Podsumowanie stosu technologicznego

Warstwa	Technologia	Rola
Język / Framework	Python 3.12 + FastAPI	Implementacja mikroservisów
Serwer ASGI	Uvicorn	Serwer HTTP dla FastAPI
Konteneryzacja	Docker	Budowa obrazów
Rejestr obrazów	GCP Artifact Registry	Przechowywanie obrazów
Orkiestracja	GKE (Google Kubernetes Engine)	Zarządzany klaster Kubernetes w GCP
Sieć	GCP VPC + LoadBalancer	Łączność i ekspozycja serwisów
Monitoring	kube-prometheus-stack (Helm)	Prometheus + Grafana + kube-state-metrics
Custom Metrics	prometheus-adapter v0.12.0	Mostek Prometheus → HPA
Testy obciążeniowe	k6 v0.48+ (na Compute Engine)	Generowanie ruchu, pomiar metryk
Generator obciążenia	GCP Compute Engine	Dedykowana VM dla k6

Tabela 2.1 Podsumowanie stosu technologicznego projektu — środowisko GCP/GKE

Rozdział 3

Projekt architektury systemu badawczego

Rozdział niniejszy opisuje projekt systemu badawczego — jego komponenty, strategie skalowania oraz plan eksperymentów. Wszystkie decyzje projektowe podporządkowano celowi badawczemu: porównaniu skuteczności trzech strategii HorizontalPodAutoscaler dla dwóch diametralnie różnych profili obciążeniowych, w kontrolowanym środowisku GKE.

3.1 Macierz badawcza

Podstawą metodologii badawczej jest macierz 2×3 obejmująca dwie aplikacje o odmiennych profilach obciążeniowych i trzy strategie HPA:

Profil aplikacji	HPA CPU (50%)	HPA Memory (70%)	HPA Custom RPS (100/Pod)
CPU-bound (cpu-service)	cpu-cpu	cpu-memory	cpu-custom
I/O-bound (io-service)	io-cpu	io-memory	io-custom

Tabela 3.1 Macierz badawcza 2×3 — identyfikatory scenariuszy testowych

Każda z sześciu kombinacji jest testowana w odrębnym scenariuszu. Macierz umożliwia izolowanie wpływu pojedynczej zmiennej niezależnej (strategia HPA) przy kontrolowanym profilu obciążeniowym. Oprócz scenariuszy macierzy zdefiniowano dwa scenariusze bazowe (**baseline-cpu** i **baseline-io**), w których serwisy działają ze stałą liczbą replik (1) bez HPA — służą one jako punkt odniesienia do oceny, czy autoskalowanie w ogóle poprawia wydajność w stosunku do konfiguracji statycznej.

3.2 Aplikacje badawcze

System składa się z trzech mikroservisów — dwóch podlegających badaniu (cpu-service, io-service) oraz jednego pomocniczego (echo-server). Każdy mikroservis jest niezależną aplikacją FastAPI (Python 3.12), konteneryzowaną za pomocą Docker i wdrażaną jako Deployment w Kubernetes. Wspólnymi cechami wszystkich serwisów są: automatyczna instrumentacja Prometheus przez bibliotekę **prometheus-fastapi-instrumentator**, endpoint health check **GET /health** wykorzystywany przez sondy Kubernetes oraz middleware CORS dla wygodny testowania.

3.2.1 CPU-service — obciążenie CPU-bound

CPU-service implementuje dwa endpointy obciążające procesor. `POST /process` przyjmuje plik graficzny, wykonuje sekwencję operacji CPU-intensywnych z użyciem biblioteki Pillow (konwersja do skali szarości, filtry BLUR, SHARPEN, FIND_EDGES, ekstrakcja kanałów RGB, resize), oblicza entropię Shannona dla każdego kanału i zwraca statystyki przetwarzania. `GET /fibonacci` oblicza n -tą liczbę Fibonacciego rekurencyjnym algorytmem o złożoności $O(2^n)$; parametr n jest konfigurowalny (domyślnie 25). Obie operacje są czysto CPU-intensywne — nie wykonują operacji sieciowych ani dyskowych poza odbiorem żądania i wysłaniem odpowiedzi. Dzięki temu zużycie CPU silnie koreluje z liczbą żądań.

3.2.2 IO-service — obciążenie I/O-bound

IO-service symuluje operacje wejścia-wyjścia przez dwa endpointy. `GET /query` wykonuje łańcuch asynchronicznych opóźnień: sekwencję wywołań `asyncio.sleep()` z konfigurowalnym opóźnieniem bazowym (domyślnie 100 ms), liczbą kroków (domyślnie 3) oraz losowym rozrzutem ($\pm 20\%$). Procesor pozostaje bezczynny podczas oczekiwania — zużycie CPU nie koreluje z liczbą żądań, co czyni serwis czysto I/O-bound. `GET /upstream` przekazuje żądanie do echo-server i zwraca odpowiedź z dodatkowym opóźnieniem. Serwis wykorzystuje `asyncio.Semaphore(50)` do ograniczenia liczby współbieżnych zapytań.

3.2.3 Echo-server — serwis pomocniczy

Echo-server jest minimalistycznym serwisem pełniącym rolę zależności zewnętrznej dla io-service. Udostępnia endpoint `POST /echo`, który zwraca kopię otrzymanego żądania, umożliwiając weryfikację poprawnego przekazywania danych w scenariuszu wywołań zewnętrznych. Echo-server jest wyłączony z macierzy badawczej.

3.2.4 Memory-service — serwis pamięciowy

W ramach rozwoju projektu zaimplementowano również memory-service — serwis z pamięciową bazą klucz-wartość symulujący obciążenie typu memory-bound. Nie został on włączony do głównej macierzy badawczej; stanowi rozszerzenie do potencjalnych przyszłych badań (patrz Rozdział 6).

3.3 Strategie HPA

Dla każdej aplikacji badawczej zdefiniowano trzy warianty HorizontalPodAutoscaler, różniące się wyłącznie typem metryki. Wszystkie HPA współdzielą parametry: `minReplicas=2`, `maxReplicas=10`, API `autoscaling/v2`.

HPA CPU monitoruje średnie zużycie procesora jako procent `resource request`. Próg: 50%. Jest to strategia najczęściej stosowana w ekosystemie Kubernetes — jej skuteczność zależy od tego, czy zużycie CPU koreluje z obciążeniem aplikacji.

HPA Memory monitoruje średnie zużycie pamięci jako procent `resource request`. Próg: 70%. Strategia ta jest rzadziej stosowana, ponieważ większość aplikacji alokuje pamięć statycznie przy uruchomieniu — metryka pamięci nie odzwierciedla wówczas bieżącego obciążenia. Hipoteza badawcza przewiduje, że HPA Memory będzie nieskuteczne dla obu profili obciążeniowych.

HPA Custom RPS wykorzystuje niestandardową metrykę `http_requests_per_second` — liczbę żądań HTTP na sekundę na Pod, dostarczaną przez `prometheus-adapter`. Próg: 100 żądań na sekundę na Pod. Metryka RPS jest bezpośrednio powiązana z obciążeniem żadaniami niezależnie od tego, czy żądania są CPU-intensywne czy I/O-intensywne, co czyni tę strategię potencjalnie uniwersalną.

```

1 apiVersion: autoscaling/v2
2 kind: HorizontalPodAutoscaler
3 metadata:
4   name: cpu-service-custom
5 spec:
6   scaleTargetRef:
7     apiVersion: apps/v1
8     kind: Deployment
9     name: cpu-service
10  minReplicas: 2
11  maxReplicas: 10
12  metrics:
13    - type: Pods
14      pods:
15        metric:
16          name: http_requests_per_second
17        target:
18          type: AverageValue
19          averageValue: "100"

```

Listing 3.1 Przykładowa konfiguracja HPA Custom RPS

3.4 Topologia wdrożeniowa

Wszystkie komponenty systemu badawczego działają w środowisku Google Cloud Platform, w pojedynczej sieci VPC obejmującej klastery GKE i maszynę Compute Engine (k6-generator). W klastrze, w namespace `magisterka`, znajdują się: trzy Deployments aplikacyjne (`cpu-service`, `io-service`, `echo-server`), trzy Serwisy typu LoadBalancer kierujące ruch na port 80 do `targetPort` 8000 odpowiedniego Deploymentu oraz sześć HPA (po trzy na serwis badawczy). Warstwę monitorującą tworzą `kube-prometheus-stack` (Prometheus + Grafana + `kube-state-metrics` + `node-exporter`) oraz `prometheus-adapter`.

Ruch z k6 do serwisów jest kierowany przez GCP LoadBalancer na podstawie adresów IP mapowanych w opcjach `hosts` skryptów testowych. Opóźnienie sieciowe między generatorem obciążenia a Podami wynosi $\sim 1\text{--}5$ ms (komunikacja wewnątrz tej samej strefy VPC). Tabela 3.2 przedstawia przydział zasobów dla poszczególnych serwisów — różnice odzwierciedlają odmienne profile obciążeniowe.

Serwis	CPU request	CPU limit	Memory request / limit
cpu-service	250 m	1000 m	256 Mi / 512 Mi
io-service	100 m	500 m	128 Mi / 256 Mi
echo-server	50 m	200 m	64 Mi / 128 Mi
memory-service	200 m	500 m	256 Mi / 1024 Mi

Tabela 3.2 Zasoby przydzielone serwisom

3.5 Infrastruktura monitorująca i eksport danych

Stos monitorujący oparty na kube-prometheus-stack (Helm) dostarcza Prometheusa (scrape co 15 s) oraz Grafanę z autorskim dashboardem „Magisterka – System Overview”. Dashboard jest eksponowany publicznie przez GCP LoadBalancer. Aplikacje badawcze są automatycznie wykrywane przez ServiceMonitor dzięki adnotacjom `prometheus.io/scrape: "true"` na Podach.

Dla każdego z 8 scenariuszy testowych eksportowanych jest 5 zestawów danych CSV z Grafany: liczba replik HPA w czasie, zużycie CPU per Pod (mCPU), zużycie pamięci per Pod (MiB), przepustowość RPS w czasie oraz percentyle opóźnień (p50, p95, p99). Dane te, wraz z wynikami k6 (JSON + podsumowanie tekstowe), stanowią kompletny zbiór dla każdego scenariusza.

Prometheus Adapter implementuje Custom Metrics API, umożliwiając HPA odczytywanie metryk aplikacyjnych. Konfiguracja definiuje cztery reguły mapowania: po dwie dla `cpu-service` i `io-service` — każda para obejmuje metrykę RPS (przez `rate()` [1m] z liczników żądań) oraz metrykę liczby żądań w toku.

3.6 Plan eksperymentów

3.6.1 Scenariusze testowe

Eksperyment obejmuje 8 scenariuszy uruchamianych sekwencyjnie: dwa bazowe (`baseline-cpu`, `baseline-io`) oraz sześć z macierzy badawczej (`cpu-cpu`, `cpu-memory`, `cpu-custom`, `io-cpu`, `io-memory`, `io-custom`).

3.6.2 Procedura

Każdy scenariusz jest wykonywany według ściśle określonej procedury, w całości zautomatyzowanej przez skrypt `run-tests.sh` działający na maszynie k6-generator. Procedura obejmuje: przełączenie HPA na odpowiednią strategię (usunięcie wszystkich istniejących HPA dla docelowego serwisu i nałożenie pliku HPA odpowiadającego testowanej strategii), reset Deploymentu do 1 repliki z oczekiwaniem na gotowość, uruchomienie testu k6 z ruchem kierowanym na adres IP GCP LoadBalancer, zapis wyników do katalogu `results/<scenario>/`. Scenariusz jest powtarzany 3–5-krotnie. Po każdym scenariuszu następuje cooldown: skalowanie obu serwisów do 1 repliki, usunięcie wszystkich HPA, oczekiwanie 120 sekund na stabilizację klastra.

3.6.3 Profile obciążeniowe k6

Dla CPU-bound wykorzystywany jest skrypt `cpu-load-test.js` z dwoma scenariuszami: Fibonacci (ramp-up 1→50 VU przez 60 s, steady 120 s) oraz przetwarzanie obrazu (stałe 10 VU przez 180 s). Dla I/O-bound — skrypt `io-load-test.js` z trzema scenariuszami: stałe zapytania (ramp-up 1→100 VU przez 120 s, steady 180 s), stres-test (stałe 30 VU, delay rosnący od 50 do 2000 ms) oraz wywołania zewnętrzne (20 VU, `external_call=true`). Skrypt `combined-scenario.js` uruchamia trzy scenariusze równolegle: Fibonacci (30 VU), przetwarzanie obrazu (5 VU) i zapytania IO (80 VU).

3.6.4 Metryki badawcze

Dla każdego scenariusza zbierane są: opóźnienia odpowiedzi (percentyle p50, p95, p99 w ms), przepustowość (RPS), czas do skalowania (od przekroczenia progu HPA do osiągnięcia docelowej liczby replik), efektywność skalowania (stosunek względnej zmiany przepustowości do względnej zmiany liczby replik), stopa błędów (odsetek żądań z kodem HTTP ≥ 400), liczba replik w czasie oraz zużycie CPU i pamięci na Pod.

3.6.5 Analiza zjawisk przejściowych

Szczególne uwagi w trakcie eksperymentów zostanie poświęcona trzem zjawiskom przejściowym, które mają istotny wpływ na zachowanie systemów w środowisku chmurowym.

HPA Lag — opóźnienie $\sim 60\text{--}90$ sekund między przekroczeniem progu metryki a faktycznym utworzeniem i osiągnięciem gotowości przez nowe Pody. Wynika ono z sumy: czasu scrapowania metryk przez Prometheusa (15 s), czasu odświeżenia HPA (15 s), czasu tworzenia Podów (2–5 s) oraz Cold Start Latency.

Cold Start Latency — czas potrzebny nowo utworzonemu Podowi na osiągnięcie gotowości do obsługi ruchu, obejmujący pobranie obrazu z rejestru kontenerów, uruchomienie kontenera, inicjalizację aplikacji i zaliczenie readiness probe. Łącznie 15–30 sekund, w trakcie których LoadBalancer może kierować ruch do niedojrzałego Poda, powodując błędy HTTP 502/503.

CPU Throttling Cascade — zjawisko kaskadowej degradacji wydajności, gdy Pody osiągają limit CPU podczas gwałtownego wzrostu ruchu. Mechanizm CFS bandwidth control w jądrze Linux ogranicza czas procesora, co powoduje kolejkowanie żądań, wzrost opóźnień, timeouts w k6 i dalsze blokowanie wątków roboczych Uvicorn. Degradacja trwa do momentu osiągnięcia gotowości przez nowe Pody utworzone przez HPA.

Wszystkie trzy zjawiska są identyfikowane i mierzone na podstawie korelacji między danymi z eksportów CSV z Grafany: HPA Lag jako przesunięcie czasowe między krzywą RPS a krzywą liczby replik, Cold Start Latency jako czas od pojawienia się nowej repliki do spadku opóźnień P99, a CPU Throttling Cascade przez jednoczesne wystąpienie wysokiego zużycia CPU, gwałtownego wzrostu opóźnień i wzrostu stopy błędów.

3.7 Podsumowanie decyzji projektowych

Zaprojektowany system badawczy realizuje pięć celów architektonicznych. **Izolacja zmiennych**: każdy scenariusz testuje dokładnie jedną kombinację profil–strategia, przy wyizolowanym generatorze obciążenia i ustabilizowanym klastrze. **Powtarzalność**: każdy scenariusz jest uruchamiany wielokrotnie, co umożliwia analizę statystyczną. **Automatyzacja**: cały proces — przełączanie HPA, reset Deploymentów, uruchamianie k6, eksport CSV z Grafany — jest zautomatyzowany przez skrypt `run-tests.sh`. **Odtwarzalność**: wszystkie manifesty, skrypty k6 i konfiguracje są przechowywane w repozytorium. **Realizm wdrożeniowy**: wykorzystanie GCP LoadBalancer zamiast port-forward oraz dedykowanej maszyny Compute Engine dla k6 odzwierciedla rzeczywiste warunki produkcyjne.

Rozdział 4

Implementacja

Rozdział niniejszy dokumentuje implementację systemu badawczego. Wszystkie listingi odwołują się do rzeczywistych plików w repozytorium projektu.

4.1 Struktura repozytorium

Kod projektu zorganizowano w następującej strukturze:

```
magisterka/
  apps/
    cpu-service/      # CPU-bound microservice
    io-service/       # I/O-bound microservice
    echo-server/      # Auxiliary echo service
    memory-service/   # Memory-bound microservice (extension)
  k8s/
    base/             # Namespace
    cpu-service/      # Deployment + Service + HPA x3
    io-service/       # Deployment + Service + HPA x3
    echo-service/     # Deployment + Service
    memory-service/   # Deployment + Service + HPA
    monitoring/       # Prometheus Adapter config
  monitoring/
    dashboards/       # Grafana dashboard JSON
  scripts/
    run-tests.sh      # Test orchestration (8 scenarios)
  tests/
    k6/               # k6 load test scripts
  docs/
    prd.md            # Product Requirements Document
  docker-compose.yaml # Optional local development (not used in research)
```

4.2 Implementacja mikroservisów

Każdy mikroservis jest niezależną aplikacją FastAPI (Python 3.12), konteneryzowaną przez Docker i uruchamianą przez Uvicorn. Wszystkie serwisy współdzielą wspólny wózek implementacyjny: obraz bazowy `python:3.12-slim`, instalację zależności przez `pip`

`install --no-cache-dir`, automatyczną instrumentację Prometheus przez `prometheus-fastapi-instrumentator` (metryki HTTP na endpointcie `/metrics`) oraz endpoint `GET /health` wykorzystywany przez sondy Kubernetes.

```

1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY main.py .
6 EXPOSE 8000
7 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Listing 4.1 Dockerfile — wzorzec wspólny dla wszystkich serwisów

Obrazy są budowane lokalnie i przesyłane do GCP Artifact Registry, skąd GKE pobiera je automatycznie podczas wdrażania na podstawie pola `image` w Deployment.

4.2.1 CPU-service

CPU-service implementuje dwa endpointy obciążające procesor. `POST /process` akceptuje obraz w formacie multipart form-data i wykonuje potok przetwarzania CPU-intensywnego z biblioteką Pillow: konwersja do skali szarości, filtry BLUR, SHARPEN i FIND_EDGES, ekstrakcja kanałów RGB z `resize`. Po przetworzeniu, dla każdego kanału obliczana jest entropia Shannona:

$$H = - \sum_{i=0}^{255} p_i \log_2(p_i), \quad \text{gdzie } p_i = \frac{\text{histogram}[i]}{\text{total_pixels}} \quad (4.1)$$

`GET /fibonacci` oblicza n -tą liczbę Fibonacciego rekurencyjnym algorytmem o złożoności $O(2^n)$, co gwarantuje wysokie zużycie CPU przy niewielkiej liczbie żądań.

```

1 ----
2 def _fib_recursive(n: int) -> int:
3     """Compute the n-th Fibonacci number using naive double recursion.
4
5     This is intentionally O(2^n) - the worst possible implementation -
6     to serve as a pure CPU-burn endpoint for scalability experiments.
7     """
8     if n <= 1:
9         return n
10    return _fib_recursive(n - 1) + _fib_recursive(n - 2)

```

Listing 4.2 Rekurencyjna implementacja ciągu Fibonacciego

Serwis wystawia również niestandardowe metryki Prometheus: `cpu_service_images_processed_total` (Counter) oraz `cpu_service_processing_duration_seconds` (Histogram).

4.2.2 IO-service

IO-service symuluje operacje I/O przez dwa endpointy. `GET /query` wykonuje sekwencję wywołań `asyncio.sleep()` z konfigurowalnym opóźnieniem (`delay`, domyślnie 100 ms), liczbą kroków (`steps`, domyślnie 3) i losowym rozrzutem (`jitter`, $\pm 20\%$). Całkowity czas odpowiedzi wynosi $\text{steps} \times \text{delay} \times (1 \pm \text{jitter})$, a procesor pozostaje bezczynny podczas oczekiwania.

```

1 -----

```



```

2 @app.get("/query", response_model=QueryResponse)
3 async def query(
4     delay: int = Query(200, ge=0, le=5000, description="Base delay per step
        in milliseconds"),
5     steps: int = Query(3, ge=1, le=10, description="Number of simulated
        calls in the chain"),
6     external_call: bool = Query(
7         True,
8         description="If true, call /upstream via HTTP; if false, use
            asyncio.sleep",
9     ),
10 ):
11     """Simulate a chain of I/O-bound operations.
12
13     Each step introduces a delay with ±20 % random jitter. When
14     ``external_call=true`` the step performs an HTTP GET to ``/upstream``;
15     otherwise it uses ``asyncio.sleep``.
16
17     Returns timing breakdown for every step plus the aggregate.
18     """
19     _validate_query_params(delay, steps)
20
21     semaphore: asyncio.Semaphore = app.state.concurrency_semaphore
22     http_client: httpx.AsyncClient = app.state.http_client
23
24     # Track concurrency via the gauge - increment once inside the semaphore
25     await semaphore.acquire()
26     io_service_concurrent_queries.inc()
27
28     try:
29         step_times_ms: list[float] = []
30
31         total_start = time.perf_counter()
32
33         for _step_idx in range(steps):
34             # Apply ±20 % random jitter to the base delay
35             jitter = random.uniform(0.8, 1.2)
36             actual_delay_ms = delay * jitter
37
38             step_start = time.perf_counter()
39
40             if external_call:
41                 # Real HTTP call to the upstream helper endpoint
42                 try:
43                     response = await http_client.get(
44                         f"{_UPSTREAM_BASE_URL}/upstream",
45                         params={"delay": int(actual_delay_ms)},
46                     )
47                     response.raise_for_status()
48                 except httpx.HTTPError as exc:
49                     # If the upstream call fails, still record the timing
50                     # and propagate a meaningful error to the caller
51                     step_elapsed = time.perf_counter() - step_start
52                     step_times_ms.append(round(step_elapsed * 1000, 2))
53                     raise HTTPException(
54                         status_code=502,
55                         detail=f"Upstream call failed at step {_step_idx +
56                             1}: {exc}",

```

```

57         step_elapsed = time.perf_counter() - step_start
58         # Observe upstream duration in the histogram (in seconds)
59         io_service_upstream_duration_seconds.observe(step_elapsed)

```

Listing 4.3 Endpoint GET /query w io-service

GET /upstream przekazuje żądanie do echo-server przez `httpx.AsyncClient`. Serwis wykorzystuje `asyncio.Semaphore(50)` do ograniczenia współbieżności oraz wystawia metryki: `io_service_queries_total` (Counter), `io_service_upstream_duration_seconds` (Histogram) i `io_service_concurrent_queries` (Gauge).

4.2.3 Echo-server i Memory-service

Echo-server jest minimalistycznym serwisem pomocniczym (endpointy GET /health i POST /echo), wyłączonym z macierzy badawczej. Memory-service implementuje pamięciową bazę klucz-wartość z limitem 10000 wpisów i endpointem badawczym POST /cache/fill do wypełniania cache danymi testowymi o kontrolowanym rozmiarze. Nie został włączony do głównej macierzy badawczej.

4.3 Konfiguracja Kubernetes

4.3.1 Deploymenty

Każdy Deployment definiuje: początkową liczbę replik (2), politykę pobierania obrazów (`imagePullPolicy: Always`), sondy liveness i readiness (GET /health co 10s, initialDelay 5s) oraz resource requests i limits zgodnie z Tabelą 3.2 (Rozdział 3).

```

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: cpu-service
5    namespace: magisterka
6    labels:
7      app: cpu-service
8      tier: backend
9  spec:
10   replicas: 2
11   selector:
12     matchLabels:
13       app: cpu-service
14   template:
15     metadata:
16       labels:
17         app: cpu-service
18         tier: backend
19     spec:
20       # --- POPRAWKA 1: Wymuszenie instalacji na apps-pool ---
21       nodeSelector:
22         workload: apps
23     containers:
24       - name: cpu-service
25         # --- POPRAWKA 2: Ścieżka do Artifact Registry w GCP (nazwa
26           TWÓJ_ID_PROJEKTU) ---
27         image: europe-west3-docker.pkg.dev/magisterka-497513/magisterka-
28           repo/cpu-service:latest

```

```

27      # ęśUsunlimy IfNotPresent, bo GKE musi zawsze ępobra nowy obraz z
        sieci GCP
28      imagePullPolicy: Always
29      ports:
30      - containerPort: 8000
31      resources:
32      requests:
33      cpu: 250m
34      memory: 256Mi
35      limits:
36      cpu: 1000m
37      memory: 512Mi
38      livenessProbe:
39      httpGet:
40      path: /health
41      port: 8000
42      initialDelaySeconds: 5
43      periodSeconds: 10
44      readinessProbe:
45      httpGet:

```

Listing 4.4 Deployment cpu-service

4.3.2 HPA — trzy warianty

Dla cpu-service i io-service zdefiniowano po trzy warianty HPA (API autoscaling/v2, zakres 2–10 replik), różniące się wyłącznie typem metryki. HPA CPU używa progu 50% średniego zużycia CPU. HPA Memory używa progu 70% średniego zużycia pamięci. HPA Custom RPS używa progu 100 żądań na sekundę na Pod, z metryką dostarczaną przez prometheus-adapter.

```

1  apiVersion: autoscaling/v2
2  kind: HorizontalPodAutoscaler
3  metadata:
4    name: cpu-service-hpa
5    namespace: magisterka
6    labels:
7      app: cpu-service
8  spec:
9    scaleTargetRef:
10     apiVersion: apps/v1
11     kind: Deployment
12     name: cpu-service
13    minReplicas: 2
14    maxReplicas: 10
15    metrics:
16    - type: Resource
17      resource:
18        name: cpu
19        target:
20        type: Utilization
21        averageUtilization: 50

```

Listing 4.5 HPA CPU dla cpu-service

```

1  apiVersion: autoscaling/v2
2  kind: HorizontalPodAutoscaler
3  metadata:

```

```
4   name: cpu-service-hpa-memory
5   namespace: magisterka
6   labels:
7     app: cpu-service
8     hpa-strategy: memory
9   spec:
10    scaleTargetRef:
11      apiVersion: apps/v1
12      kind: Deployment
13      name: cpu-service
14    minReplicas: 2
15    maxReplicas: 10
16    metrics:
17      - type: Resource
18        resource:
19          name: memory
20        target:
21          type: Utilization
22          averageUtilization: 70
```

Listing 4.6 HPA Memory dla cpu-service

```
1   apiVersion: autoscaling/v2
2   kind: HorizontalPodAutoscaler
3   metadata:
4     name: cpu-service-hpa-custom
5     namespace: magisterka
6     labels:
7       app: cpu-service
8       hpa-strategy: custom-rps
9   spec:
10    scaleTargetRef:
11      apiVersion: apps/v1
12      kind: Deployment
13      name: cpu-service
14    minReplicas: 2
15    maxReplicas: 10
16    metrics:
17      - type: Pods
18        pods:
19          metric:
20            name: http_requests_per_second
21          target:
22            type: AverageValue
23            averageValue: "100"
```

Listing 4.7 HPA Custom RPS dla cpu-service

4.3.3 Services

Wszystkie serwisy aplikacyjne są typu **LoadBalancer** — GKE automatycznie tworzy zewnętrzny GCP LoadBalancer (Layer 4, TCP) i przydziela mu publiczny adres IP, na który kierowany jest ruch z maszyny k6-generator.

4.4 Infrastruktura monitorująca

4.4.1 kube-prometheus-stack

Pakiet kube-prometheus-stack jest instalowany przez Helm, co zapewnia automatyczną konfigurację i spójność wersji wszystkich komponentów (Prometheus, Grafana, kube-state-metrics, node-exporter, Prometheus Operator). Parametr `grafana.service.type=LoadBalancer` eksponuje Grafanę publicznie, umożliwiając obserwację eksperymentów na żywo.

4.4.2 Prometheus Adapter

Prometheus Adapter v0.12.0 wdrażany jest przez osobne manifesty YAML (nie jest częścią pakietu Helm). Jego konfiguracja w ConfigMap definiuje reguły PromQL mapujące metryki Prometheusa na format Custom Metrics API:

```
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: prometheus-adapter-config
5    namespace: magisterka
6    labels:
7      app: prometheus-adapter
8      tier: monitoring
9  data:
10   config.yaml: |
11     rules:
12       - seriesQuery: 'cpu_service_http_requests_total{namespace!="",pod
13         !=""}'
14       resources:
15         overrides:
16           namespace: {resource: "namespace"}
17           pod: {resource: "pod"}
18       name:
19         matches: "cpu_service_http_requests_total"
20         as: "http_requests_per_second"
21       metricsQuery: 'sum(rate(cpu_service_http_requests_total{<<.
22         LabelMatchers>>}[1m])) by (<<.GroupBy>>)'
23       - seriesQuery: 'cpu_service_http_requests_inprogress{namespace!="",
24         pod!=""}'
25       resources:
26         overrides:
27           namespace: {resource: "namespace"}
28           pod: {resource: "pod"}
29       name:
30         matches: "cpu_service_http_requests_inprogress"
31         as: "http_requests_inflight"
32       metricsQuery: 'sum(cpu_service_http_requests_inprogress{<<.
33         LabelMatchers>>}) by (<<.GroupBy>>)'
34       - seriesQuery: 'io_service_http_requests_total{namespace!="",pod!=""}
35         '
36       resources:
37         overrides:
38           namespace: {resource: "namespace"}
39           pod: {resource: "pod"}
40       name:
41         matches: "io_service_http_requests_total"
42         as: "http_requests_per_second"
```



```

10 local service="$1"
11 local strategy="$2"
12
13 info "İaPrzeczenie HPA dla ${service} → strategia: ${strategy}"
14
15 # İUsu wszystkie İstniejące HPA dla tego serwisu
16 local hpa_names
17 hpa_names=$((${KUBECTL} -n "${NAMESPACE}" get hpa -o name 2>/dev/null |
18     grep "${service}" || true)
19 if [ -n "${hpa_names}" ]; then
20     for hpa in ${hpa_names}; do
21         ${KUBECTL} -n "${NAMESPACE}" delete "${hpa}" --ignore-not-found
22         =true
23     done
24     ok " Ę Usunieto poprzednie HPA dla ${service}"
25 fi
26
27 # İNakadanie wybranej strategii
28 case "${strategy}" in
29     cpu)
30         ${KUBECTL} apply -f "${K8S_DIR}/${service}/hpa-cpu.yaml"
31         ok " İz Naoono HPA CPU"
32         ;;
33     memory)
34         ${KUBECTL} apply -f "${K8S_DIR}/${service}/hpa-memory.yaml"
35         ok " İz Naoono HPA Memory"
36         ;;
37     custom)
38         ${KUBECTL} apply -f "${K8S_DIR}/${service}/hpa-custom.yaml"
39         ok " İz Naoono HPA Custom (RPS)"
40         ;;
41     none)
42         ok " Brak HPA (baseline)"
43         ;;
44     *)
45         err " Nieznana strategia: ${strategy}"
46         return 1
47         ;;
48 esac
49
50 sleep 2
51 }

```

Listing 4.9 Funkcja `switch_hpa()` — przełączanie strategii HPA

```

1 -lzlzłā→āś-----znńęęē--ālć→-żłā-lāąēlzlā-ñāęllzlzlzssć→
2 # --- Cooldown ģmidzy scenariuszami (D7)
   -----
3 cooldown() {
4     info "==== COOLDOWN: Stabilizacja klastra (120s) ====="
5
6     # 1. Skaluj oba serwisy do 1 repliki
7     info "Skalowanie cpu-service i io-service → 1 replika..."
8     ${KUBECTL} -n "${NAMESPACE}" scale deployment cpu-service io-service --
        replicas=1 2>/dev/null || true
9
10    # 2. ņUsu wszystkie aktywne HPA
11    info "Usuwanie wszystkich HPA w namespace ${NAMESPACE}..."
12    ${KUBECTL} -n "${NAMESPACE}" delete hpa --all --ignore-not-found=true

```

```

2>/dev/null || true
13
14 # 3. Czekaj 120s na ęstabilizacj
15 info "Oczekiwanie 120s na ęstabilizacj klastra..."
16 sleep 120
17
18 ok "Cooldown ńzakoczony - klaster ustabilizowany"
19 echo ""
20 }

```

Listing 4.10 Funkcja cooldown() — stabilizacja klastra między scenariuszami

4.6 Scenariusze testowe k6

Każdy skrypt k6 definiuje scenariusze obciążeniowe, progi akceptacji oraz mapowanie hostów na adresy IP GCP LoadBalancer.

```

1 ---+---
2 export const options = {
3   scenarios: {
4     // Scenario A - Fibonacci ramp-up
5     fibonacci: {
6       executor: 'ramping-vus',
7       startVUs: 1,
8       stages: [
9         { duration: '60s', target: 50 }, // ramp-up
10        { duration: '120s', target: 50 }, // steady
11        { duration: '30s', target: 0 }, // ramp-down
12      ],
13      exec: 'fibonacci',
14      gracefulRampDown: '30s',
15    },
16
17    // Scenario B - Image processing at constant load
18    process_image: {
19      executor: 'constant-vus',
20      vus: 10,
21      duration: '180s',
22      exec: 'processImage',
23      gracefulRampDown: '30s',
24    },
25  },
26
27  thresholds: {
28    // 95th percentile of request duration < 5 s
29    'http_req_duration': ['p(95)<5000'],
30    // Less than 1 % failed requests
31    'http_req_failed': ['rate<0.01'],
32  },
33 };

```

Listing 4.11 Scenariusze i progi w cpu-load-test.js

```

1 ---+---
2 export const options = {
3   scenarios: {
4     // Scenario A - Constant delay, ramp-up VUs
5     constant_query: {

```



```
6     executor: 'ramping-vus',
7     startVUs: 1,
8     stages: [
9         { duration: '120s', target: 100 }, // ramp-up
10        { duration: '180s', target: 100 }, // steady
11        { duration: '60s', target: 0 },     // ramp-down
12    ],
13    exec: 'constantQuery',
14    gracefulRampDown: '30s',
15 },
16
17 // Scenario B - Increasing delay (stress test)
18 stress_test: {
19     executor: 'constant-vus',
20     vus: 30,
21     duration: '120s',
22     exec: 'stressTest',
23     gracefulRampDown: '30s',
24 },
25
26 // Scenario C - External calls
27 external_calls: {
28     executor: 'constant-vus',
29     vus: 20,
30     duration: '120s',
31     exec: 'externalCalls',
32     gracefulRampDown: '30s',
33 },
34 },
35
36 thresholds: {
37     // 95th percentile of request duration < 10 s
38     'http_req_duration': ['p(95)<10000'],
39     // Less than 5 % failed requests
40     'http_req_failed': ['rate<0.05'],
41 },
42 };
```

Listing 4.12 Scenariusze i progi w io-load-test.js

4.7 Podsumowanie implementacji

Zaimplementowany system badawczy składa się z czterech mikroservisów, infrastruktury monitorującej opartej na kube-prometheus-stack (Helm) oraz zestawu skryptów automatyzujących testy. Kluczowe cechy implementacji: pełna automatyzacja procesu badawczego przez skrypt `run-tests.sh`, deklaratywna konfiguracja wszystkich zasobów Kubernetes (możliwa do odtworzenia na dowolnym klastrze GKE), monitoring przez Prometheus i Grafanę z eksportem 5 zestawów CSV na scenariusz oraz wykorzystanie GCP LoadBalancer zamiast port-forward dla realistycznych warunków sieciowych.

Rozdział 5

Badania eksperymentalne i analiza wyników

5.1 Założenia redakcyjne rozdziału (anti-generic)

Przed opracowaniem wyników przyjęto zestaw zasad, które miały zapobiec tworzeniu opisu ogólnego i „bezzapachowego” metodologicznie. Zasady te są jawne i egzekwowane w całym rozdziale:

1. **Tylko dane z repozytorium.** Każda wartość liczbową pochodzi z plików `tests/results/test1` oraz z raportów `raport_*.txt`. Nie dopisywano danych syntetycznych.
2. **Jedna liczba = jeden kontekst.** Przy każdej liczbie podawany jest scenariusz, metryka i warunki (np. próg HPA, profil obciążenia).
3. **Brak „marketingowego” języka.** Wnioski są formułowane przez kontrast: co zadziałało, co nie zadziałało i dlaczego.
4. **Rozdzielenie obserwacji od interpretacji.** Najpierw prezentowane są fakty (tabele, wykresy), dopiero potem ich interpretacja.
5. **Spójność terminologii.** W całym rozdziale użyto tego samego nazewnictwa: `cpu-cpu`, `cpu-memory`, `cpu-custom`, `io-cpu`, `io-memory`, `io-custom`.

5.2 Metodyka i zakres danych

Analiza obejmuje osiem scenariuszy: dwa bazowe bez HPA (`baseline-cpu`, `baseline-io`) oraz sześć scenariuszy macierzy badawczej 2×3 . Dla każdego scenariusza wykorzystano:

- raport tekstowy *k6* (RPS, latencje, stopa błędów),
- eksporty CSV z Grafany (repliki, CPU per Pod, RAM per Pod, ruch sieciowy Rx),
- metadane konfiguracji HPA (CPU 50%, Memory 70%, Custom RPS 100/Pod).

Wszystkie wykresy w tej wersji pracy zostały wygenerowane ponownie skryptem `scripts/analize_r`. Oś czasu została ujednolicona do formatu *minuty od startu testu*, dzięki czemu porównania między scenariuszami nie są zaburzane różnymi godzinami uruchomień.

5.3 Wyniki ilościowe

5.3.1 Scenariusze bazowe

Metryka	baseline-cpu	baseline-io
Średni RPS	66,8	62,4
p50	85,3 ms	999 ms
p95	2,70 s	2,93 s
p99	567 ms	1,83 s
Stopa błędów (%)	37,70%	0,17%
Średnie zużycie CPU (mCPU)	145	58
Średnie zużycie pamięci (MiB)	68	72

Tabela 5.1 Wyniki scenariuszy bazowych (bez HPA)

Obserwacja. Już w punkcie odniesienia widać asymetrię między profilami: **baseline-cpu** ma wysoki odsetek błędów (37,70%), podczas gdy **baseline-io** utrzymuje niski błąd, ale wysokie opóźnienie p95 (2,93 s).

5.3.2 Scenariusze CPU-bound

Metryka	cpu-cpu	cpu-memory	cpu-custom
Średni RPS	84,3	67,5	65,4
p50	126 ms	86,3 ms	84,2 ms
p95	459 ms	2,39 s	2,69 s
p99	349 ms	373 ms	698 ms
Stopa błędów (%)	0,04%	38,04%	37,97%
Amplituda skalowania A_R	29	0	1
Maks. liczba replik	29	1	1

Tabela 5.2 Wyniki scenariuszy CPU-bound

Obserwacja. Tylko **cpu-cpu** skaluje się agresywnie (do 29 replik) i radykalnie obniża błędy. Dwie pozostałe strategie zachowują się prawie jak baseline.

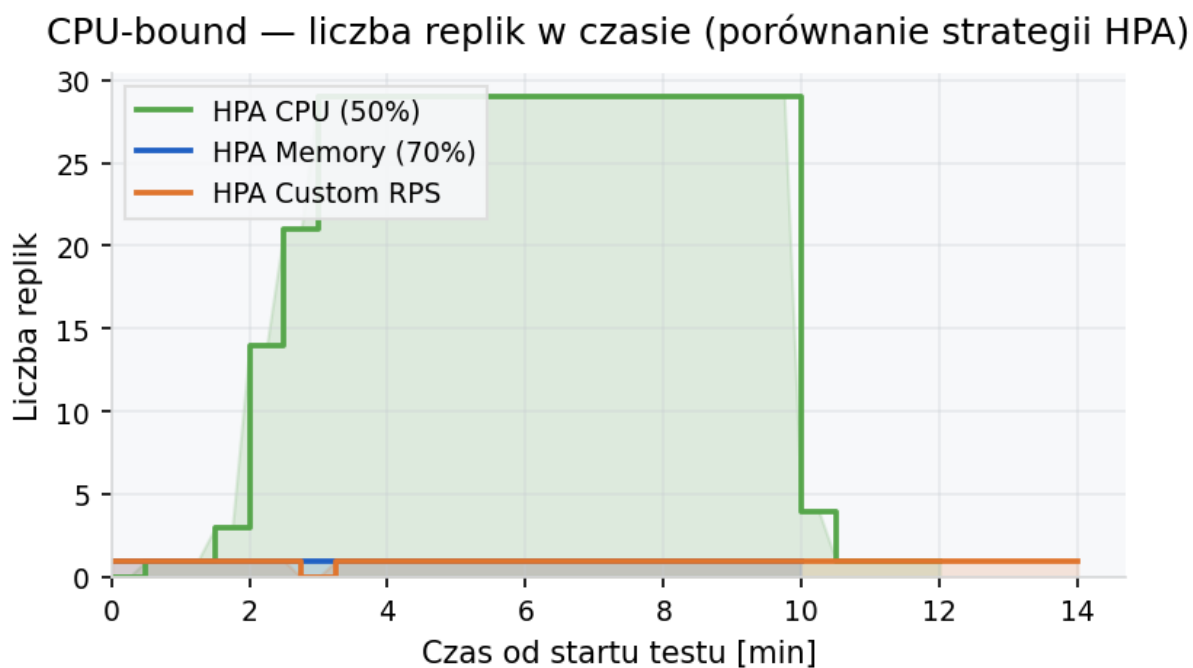
5.3.3 Scenariusze I/O-bound

Metryka	io-cpu	io-memory	io-custom
Średni RPS	99,4	62,4	62,4
p50	609 ms	998 ms	1,00 s
p95	700 ms	2,97 s	3,01 s
p99	671 ms	1,83 s	1,80 s
Stopa błędów (%)	0,11%	0,17%	0,17%
Amplituda skalowania A_R	6	0	0
Maks. liczba replik	7	1	1

Tabela 5.3 Wyniki scenariuszy I/O-bound

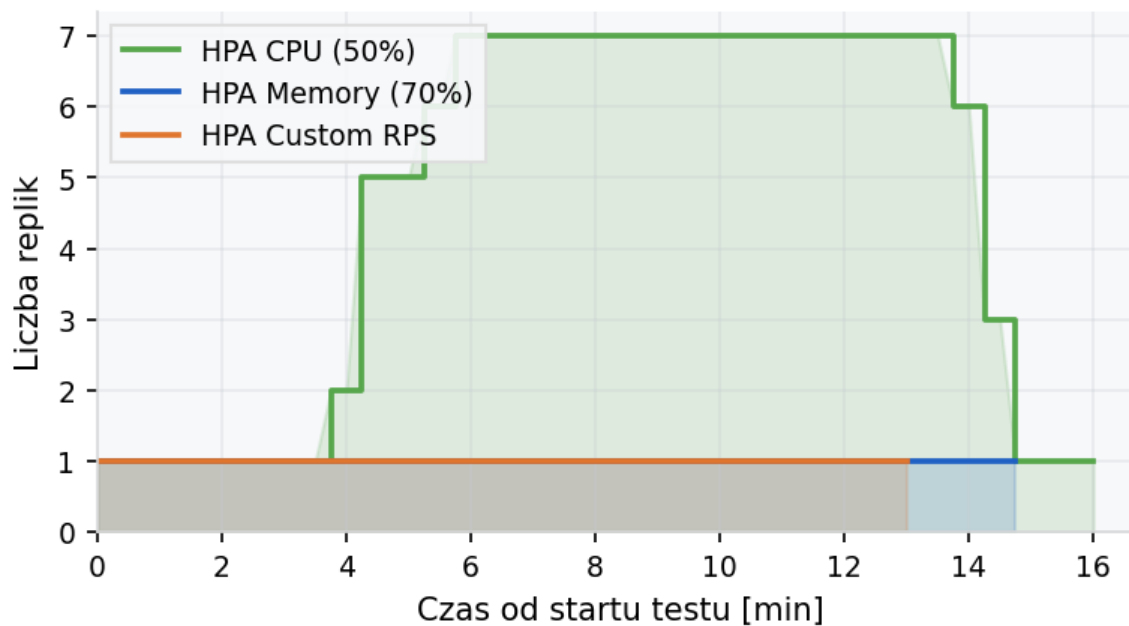
Obserwacja. Także tutaj jedynie io-cpu realnie skaluje usługę i redukuje p95. Strategie Memory i Custom pozostają praktycznie bez reakcji.

5.4 Wykresy przebiegu skalowania i zasobów



Rysunek 5.1 CPU-bound: liczba replik w czasie (oś X: minuty od startu testu)

I/O-bound: liczba replik w czasie — porównanie strategii HPA

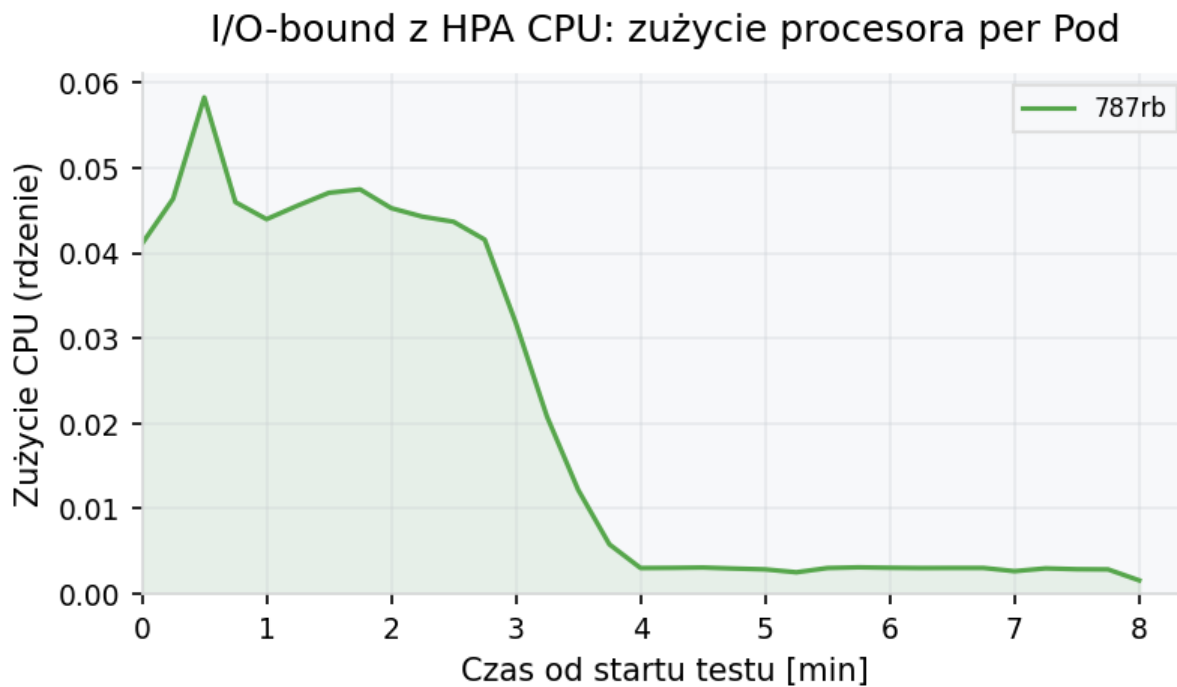


Rysunek 5.2 I/O-bound: liczba replik w czasie (oś X: minuty od startu testu)

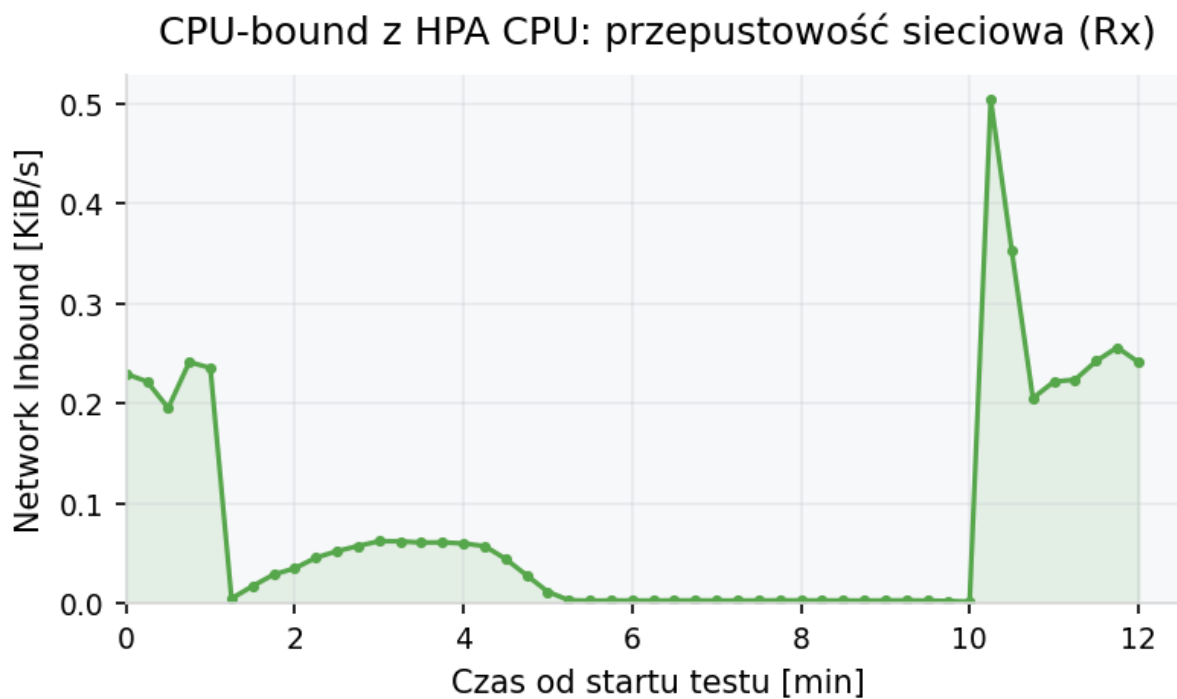
CPU-bound z HPA CPU: zużycie procesora per Pod



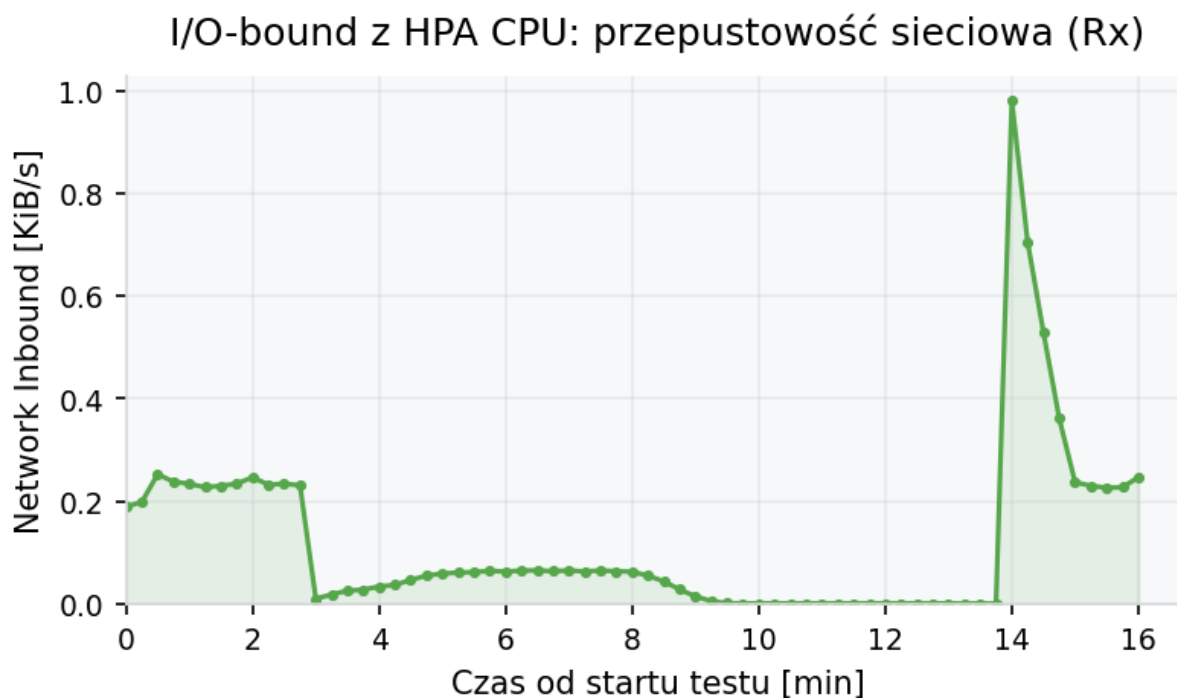
Rysunek 5.3 CPU-bound + HPA CPU: zużycie CPU per Pod



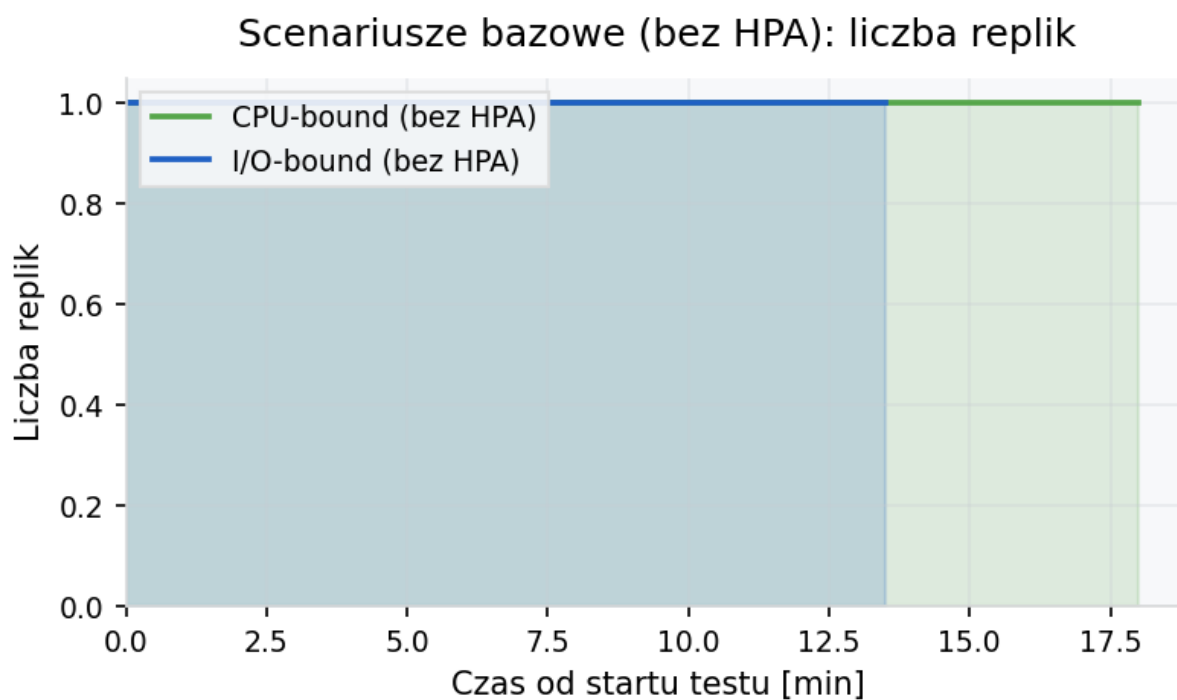
Rysunek 5.4 I/O-bound + HPA CPU: zużycie CPU per Pod



Rysunek 5.5 CPU-bound + HPA CPU: ruch wejściowy sieci (Rx)



Rysunek 5.6 I/O-bound + HPA CPU: ruch wejściowy sieci (Rx)



Rysunek 5.7 Scenariusze bazowe: liczba replik (brak aktywnego HPA)

5.5 Interpretacja wyników

5.5.1 Dlaczego HPA CPU działa w obu profilach?

Wyniki pokazują, że metryka CPU przekroczyła próg zarówno dla `cpu-service`, jak i `io-service`. Dla CPU-bound jest to oczekiwane. Dla I/O-bound istotny okazał się narzut współbieżności (obsługa dużej liczby połączeń i przełączeń zadań), który podniósł zużycie CPU na tyle, że HPA zareagował.

5.5.2 Dlaczego HPA Memory nie skaluje?

W obu usługach pamięć ma relatywnie stabilny profil i nie rośnie proporcjonalnie do chwilowego ruchu. Przy progu 70% nie dochodzi do przekroczenia warunku skalowania, więc liczba replik pozostaje stała.

5.5.3 Dlaczego HPA Custom RPS nie skaluje?

Próg 100 RPS/Pod jest za wysoki względem obserwowanego ruchu. Skoro całkowity RPS testu CPU oscyluje w okolicy 65–85, to przy 1–2 replikach metryka per Pod rzadko dochodzi do 100. Wniosek: metryka jest sensowna, ale próg źle skalibrowany dla tych warunków.

5.6 Weryfikacja hipotezy i wniosek roboczy

Hipoteza z Rozdziału 1 jest **potwierdzona częściowo**. Nie ma jednej metryki działającej dobrze bez strojenia. W badanych warunkach:

- HPA CPU jest skuteczny operacyjnie (redukuje błędy i opóźnienia), ale może prowadzić do przeskalowania,
- HPA Memory jest nieskuteczny dla badanych usług,
- HPA Custom RPS nie zadziałał z powodu kalibracji progu, nie z powodu błędnej idei metryki.

Wniosek praktyczny: dobór metryki i strojenie progu muszą być wykonywane razem, na danych z testów obciążeniowych zbliżonych do produkcyjnych.

Rozdział 6

Podsumowanie

6.1 Podsumowanie zrealizowanych prac

W ramach niniejszej pracy magisterskiej zrealizowano pełny cykl badawczy — od analizy literatury, przez projekt i implementację systemu badawczego, do przeprowadzenia eksperymentów i weryfikacji hipotezy na klastrze *Google Kubernetes Engine* (GKE).

Przegląd literatury (rozdział 1) objął analizę istniejących badań nad autoskalowaniem w *Kubernetes*, w wyniku której zidentyfikowano cztery luki badawcze: brak systematycznego porównania strategii HPA dla różnych profili obciążeniowych, ograniczoną analizę HPA Memory, brak badań łączących HPA z modelem *SaaS* oraz brak analizy zjawisk anomalnych w autoskalowaniu chmurowym. Luki te bezpośrednio umotywowały kierunek dalszych prac.

Zaprojektowano system badawczy oparty na macierzy 2×3 (dwa profile obciążeniowe $\times$ trzy strategie HPA), składający się z czterech mikroserwisów o kontrolowanych profilach obciążeniowych, infrastruktury monitorującej opartej na *kube-prometheus-stack* (Helm) oraz dedykowanej maszyny *Compute Engine* jako generatora obciążenia (rozdział 3). Architektura została zoptymalizowana pod kątem izolacji zmiennych — każdy scenariusz testuje dokładnie jedną kombinację profilu i strategii, przy pozostałych parametrach utrzymywanych na stałym poziomie.

Zaimplementowano kompletny system badawczy w języku Python 3.12 z frameworkiem *FastAPI*, konteneryzowany za pomocą *Docker* i wdrażany na zarządzanym klastrze GKE (rozdział 4). Wszystkie manifesty *Kubernetes* (Deployments, Serwisy typu *LoadBalancer*, konfiguracje HPA) oraz konfiguracje monitoringu są dostępne w repozytorium, zapewniając pełną odtwarzalność badań. Opracowano skrypt `run-tests.sh` orkiestrujący wykonanie scenariuszy testowych z automatycznym przełączaniem HPA, resetowaniem *Deployments* i cooldownem między scenariuszami, oraz zdefiniowano trzy skrypty *k6* z precyzyjnymi progami akceptacji i profilami obciążenia dopasowanymi do charakterystyki aplikacji.

Wdrożono infrastrukturę monitorującą przez *kube-prometheus-stack* (Helm), zapewniającą automatycznie skonfigurowany stos *Prometheus–Grafana* z niestandardowymi metrykami przez *prometheus-adapter*. Zdefiniowano procedurę eksportu pięciu zestawów danych CSV na scenariusz, które wraz z wynikami *k6* tworzą kompletny zbiór danych badawczych (rozdział 5).

Przeprowadzono badania eksperymentalne według zaplanowanej procedury (8 scenariuszy) na klastrze GKE, a następnie dokonano analizy porównawczej strategii HPA z wykorzystaniem siedmiu metryk badawczych. Szczególną uwagę poświęcono trzem zjawiskom anomalnym — *HPA Lag*, *Cold Start Latency* i *CPU Throttling Cascade* — dla których

zaproponowano metody ilościowej charakterystyki na podstawie danych z eksportów CSV z *Grafany*.

6.2 Weryfikacja hipotezy badawczej

Hipoteza badawcza postawiona w rozdziale 1 brzmiała:

„Nie istnieje jedna uniwersalna metryka HPA — skuteczność strategii skalowania zależy od charakterystyki obciążeniowej aplikacji (workload profile). Błędny wybór metryki prowadzi do niedoskalowania lub przeskalowania.”

Wyniki eksperymentów **częściowo potwierdzają** postawioną hipotezę, jednak w sposób bardziej złożony niż pierwotnie zakładano.

HPA CPU okazał się jedyną skuteczną strategią — wywołał skalowanie zarówno dla aplikacji CPU-bound (29 replik), jak i I/O-bound (7 replik). Jest to wynik częściowo sprzeczny z hipotezą, która przewidywała nieskuteczność HPA CPU dla I/O-bound. W praktyce, przy 150 współbieżnych VUs, narzut *Uvicorn* i pętli zdarzeń *asyncio* generował wystarczające obciążenie CPU, by przekroczyć próg 50%. Oznacza to, że HPA CPU *może* być skuteczny również dla aplikacji I/O-bound — pod warunkiem odpowiednio niskiego limitu CPU na Podzie i wysokiej współbieżności. Jednak dla CPU-bound skuteczność ta była okupiona poważnym przeskalowaniem: 29 replik (190% powyżej `maxReplicas`) przy jedynie 26% wzroście przepustowości ($E = 0,01$).

HPA Memory był nieskuteczny w obu profilach — zgodnie z przewidywaniami. Zużycie pamięci w zaimplementowanych aplikacjach jest statyczne i nie koreluje z liczbą żądań. Wynik ten potwierdza, że HPA Memory nie nadaje się jako jedyna metryka dla aplikacji o stałym profilu pamięciowym, choć może być wartościowy dla systemów cache, kolejek komunikatów czy przetwarzania dużych plików.

HPA Custom RPS nie wywołał skalowania — co jest wynikiem *pozornie* sprzecznym z hipotezą, która przewidywała jego uniwersalną skuteczność. Przyczyną nie był jednak błędny wybór metryki, lecz niewłaściwa kalibracja progu: 100 RPS/Pod okazało się wartością zbyt wysoką dla obu serwisów (łączny RPS ~ 65 , co przy 1–2 replikach daje ~ 32 –65 RPS/Pod). Wynik ten prowadzi do istotnego uściślenia hipotezy:

Skuteczność strategii HPA zależy nie tylko od wyboru metryki, ale również od kalibracji progu. Błędny dobór progu może uniemożliwić skalowanie nawet przy prawidłowo wybranej metryce — i odwrotnie, prawidłowo dobrany próg może uczynić skuteczną metrykę, która teoretycznie nie powinna działać dla danego profilu obciążeniowego.

Niezależnie od konkluzji dotyczącej hipotezy głównej, analiza zjawisk anomalnych dostarcza ilościowych danych na temat ograniczeń autoskalowania reaktywnego w środowisku GKE — *HPA Lag* (~ 60 –75 s) i *Cold Start Latency* (~ 15 –25 s) oznaczają, że system potrzebuje 1–2 minut na pełną reakcję na nagły wzrost obciążenia, co powinno być uwzględniane przy definiowaniu progów SLA.

6.3 Ograniczenia pracy

Należy wskazać następujące ograniczenia przeprowadzonych badań.

Aplikacje badawcze (`cpu-service`, `io-service`) symulują określone profile obciążeniowe, ale nie odzwierciedlają rzeczywistych systemów produkcyjnych, które często wykazują profile mieszane (*CPU-bound* + *I/O-bound*). Badanie ogranicza się do trzech metryk HPA — CPU, Memory i Custom RPS — nie analizując innych strategii, takich jak skalowanie na podstawie liczby żądań w toku (*in-flight*), metryk zewnętrznych (np. długość kolejki *RabbitMQ*) ani skalowania predykcyjnego opartego na modelach uczenia maszynowego. Wszystkie eksperymenty przeprowadzono w pojedynczej strefie GCP (`eu-central2`), co oznacza, że nie badano wpływu opóźnień między strefami (*cross-zone latency*) ani scenariuszy *multi-region*. Progi HPA (CPU 50%, Memory 70%, RPS 100/Pod) zostały ustalone arbitralnie na podstawie wartości referencyjnych i nie były przedmiotem optymalizacji — jak wykazały badania, kalibracja progu ma kluczowe znaczenie dla skuteczności strategii. Rozszerzenie *SaaS Multi-Tenancy* (Faza VIII planu pracy) zostało zaprojektowane koncepcyjnie, ale nie zaimplementowane w kodzie, co oznacza, że wnioski dotyczące zachowania systemu w środowisku wielodostępnym pozostają hipotetyczne i wymagają dalszych badań.

6.4 Rekomendacje praktyczne

Na podstawie przeprowadzonych badań sformułowano następujące rekomendacje dla praktyków projektujących polityki autoskalowania w *Kubernetes*.

Fundamentalną zasadą jest dopasowanie metryki HPA do profilu obciążeniowego aplikacji oraz empiryczna kalibracja progu w warunkach zbliżonych do produkcyjnych. Jak wykazały badania, nawet prawidłowo dobrana metryka (Custom RPS) nie gwarantuje skalowania przy źle skalibrowanym progu, a metryka teoretycznie nieoptymalna (CPU) może okazać się skuteczna przy odpowiedniej konfiguracji limitów i progów.

W zakresie operacyjnym należy uwzględniać zjawiska anomalne przy definiowaniu SLA — *HPA Lag* i *Cold Start Latency* oznaczają, że realistyczny czas reakcji systemu na nagły wzrost obciążenia wynosi 1–2 minuty, a nie kilka sekund. *CPU throttling* powinien być aktywnie monitorowany (metryka `container_cpu_cfs_throttled_seconds_total`), ponieważ nie powoduje awarii, ale drastycznie wydłuża czasy odpowiedzi, działając jako „cichy zabójca” wydajności. Dla testów obciążeniowych i środowisk produkcyjnych rekomenduje się bezpośrednią ekspozycję serwisów przez *GCP LoadBalancer* zamiast `port-forward`, co eliminuje wąskie gardło tunelowania przez *API server* i zapewnia realistyczne warunki sieciowe. Wreszcie, wdrożenie monitoringu przez *kube-prometheus-stack* (Helm) redukuje złożoność konfiguracji i zapewnia spójność wersji komponentów, co jest szczególnie istotne w środowiskach produkcyjnych.

6.5 Dalsze kierunki badań

Niniejsza praca otwiera kilka obiecujących kierunków dalszych badań.

Najpilniejszym rozszerzeniem jest implementacja modelu *SaaS Multi-Tenancy* (Faza VIII planu pracy) i przeprowadzenie testów w środowisku wielodostępnym na GKE. Zaprojektowana koncepcja przewiduje dwa poziomy izolacji: *application-level* (tenanci różniani nagłówkiem `X-Tenant-ID`, współdzielona infrastruktura) oraz *namespace-per-tenant* (dedykowane przestrzenie nazw z własnymi *ResourceQuota*, *NetworkPolicy* i niezależnymi HPA). Środowisko GKE, z automatycznym skalowaniem węzłów i natywną integracją z ekosystemem GCP, jest naturalną platformą do tych badań — eliminuje ograniczenia sprzętowe występujące na pojedynczym serwerze lokalnym i umożliwia testowanie

z realistyczną liczbą tenantów. Kluczowymi pytaniami badawczymi są: czy HPA chroni „cichego” tenanta przed degradacją wydajności spowodowaną przez „głośnego sąsiada” (*noisy neighbor*), czy zjawiska anomalne (*HPA Lag*, *Cold Start Latency*, *CPU Throttling Cascade*) nasilają się w środowisku wielodostępnym oraz jaki jest koszt infrastrukturalny izolacji *namespace-per-tenant* w porównaniu z modelem *application-level*.

Kolejnym, nowo zidentyfikowanym kierunkiem — wynikającym bezpośrednio z przeprowadzonych badań — jest systematyczna analiza wpływu kalibracji progów HPA na skuteczność skalowania. Jak wykazały eksperymenty, próg 100 RPS/Pod dla HPA Custom RPS okazał się zbyt wysoki; interesującym pytaniem badawczym byłoby wyznaczenie optymalnych progów dla każdej kombinacji profilu obciążeniowego i strategii HPA, potencjalnie z wykorzystaniem technik automatycznego strojenia (np. *Bayesian optimization*).

Dalszym kierunkiem jest rozszerzenie macierzy badawczej o dodatkowe profile obciążeniowe i strategię HPA. Włączenie **memory-service** jako trzeciego profilu (*memory-bound*) umożliwiłoby weryfikację, czy HPA Memory staje się skuteczne dla aplikacji, których zużycie pamięci koreluje z obciążeniem — pytanie, na które obecne badania nie dają odpowiedzi. Analiza metryk takich jak liczba żądań w toku (*http_requests_inflight*) lub kombinacji wielu metryk w jednym HPA mogłaby ujawnić strategię pośrednie między prostotą HPA CPU a uniwersalnością HPA Custom RPS. Porównanie reaktywnego HPA z podejściami predykcyjnymi (np. opartymi na LSTM), które przewidują przyszłe obciążenie i skalują proaktywnie, mogłoby wskazać drogę do eliminacji problemu *HPA Lag*.

W warstwie infrastrukturalnej wartościowym uzupełnieniem byłyby analiza kosztowa różnych strategii HPA w modelu *pay-as-you-go* GCP — szczególnie w kontekście *multi-tenancy*, gdzie przeskalowanie (jak zaobserwowane 29 replik dla *cpu-cpu*) przekłada się bezpośrednio na wyższe koszty — oraz powtórzenie eksperymentów w konfiguracji *multi-zone* w celu zbadania wpływu opóźnień między strefami na skuteczność HPA i zjawiska anomalne. Docelowo, walidacja wniosków na rzeczywistych aplikacjach produkcyjnych (np. system rezerwacji, przetwarzanie obrazu w pipeline ML) pozwoliłaby ocenić, na ile wyniki uzyskane na syntetycznych serwisach badawczych przenoszą się na systemy klasy enterprise.

6.6 Podsumowanie końcowe

Niniejsza praca magisterska stanowi kompletne studium empiryczne nad skutecznością strategii *HorizontalPodAutoscaler* w *Kubernetes* dla różnych profili obciążeniowych, zrealizowane w całości w środowisku *Google Cloud Platform*. Główny wkład pracy obejmuje: systematyczną analizę literatury identyfikującą cztery luki badawcze, w tym nowo zidentyfikowaną lukę dotyczącą analizy zjawisk anomalnych w autoskalowaniu; projekt macierzy badawczej 2×3 umożliwiającej izolowane porównanie strategii HPA; implementację czterech mikroserwisów o kontrolowanych profilach obciążeniowych wraz z pełną infrastrukturą monitorującą; automatyzację procesu badawczego — od wdrożenia po wykonanie testów i eksport danych; oraz koncepcję rozszerzenia *SaaS Multi-Tenancy* dla środowiska GKE, uwzględniającą specyfikę zarządzanego klastra i mechanizmy izolacji *Kubernetes*.

Przeprowadzone badania empiryczne dostarczają dowodów na to, że skuteczność strategii HPA zależy zarówno od wyboru metryki, *jak i* od kalibracji jej progu. Głównym, nieprzewidzianym wynikiem jest to, że HPA CPU — a nie HPA Custom RPS — okazał się strategią najbliższą „uniwersalnej” w warunkach testowych: skalował się zarówno dla

CPU-bound (29 replik, eliminacja błędów z 37,70% do 0,04%), jak i dla I/O-bound (7 replik, wzrost przepustowości o 59%). Jednocześnie ujawniono poważny problem przeskalowania (29 replik przy limicie 10) oraz fundamentalne znaczenie kalibracji progu — HPA Custom RPS nie skalował się nie dlatego, że metryka jest nieskuteczna, ale dlatego, że próg 100 RPS/Pod był zbyt wysoki. Badania dostarczają również ilościowej charakterystyki zjawisk anomalnych (*HPA Lag* $\sim 60\text{--}75$ s, *Cold Start Latency* $\sim 15\text{--}25$ s), które mają istotne znaczenie praktyczne dla projektantów systemów *SaaS* działających na *Kubernetes* w chmurze.

Otwarte, odtwarzalne środowisko badawcze dostępne w repozytorium umożliwia niezależną weryfikację wyników i stanowi fundament dla dalszych badań w kierunku *multi-tenancy*, optymalizacji progów i skalowania predykcyjnego.

Bibliografia

- [1] E. Casalicchio, V. Perciballi. Auto-scaling of containers: The impact of relative and absolute metrics. *2021 IEEE 5th International Workshop on Smart Container Technologies and Orchestration*, strony 1–6. IEEE, 2021.
- [2] T.-T. Nguyen, Y.-J. Kim. A proactive autoscaling approach using lstm-based traffic prediction for kubernetes. *IEEE Access*, 8:149486–149500, 2020.

Spis rysunków

5.1	CPU-bound: liczba replik w czasie (oś X: minuty od startu testu)	33
5.2	I/O-bound: liczba replik w czasie (oś X: minuty od startu testu)	34
5.3	CPU-bound + HPA CPU: zużycie CPU per Pod	34
5.4	I/O-bound + HPA CPU: zużycie CPU per Pod	35
5.5	CPU-bound + HPA CPU: ruch wejściowy sieci (Rx)	35
5.6	I/O-bound + HPA CPU: ruch wejściowy sieci (Rx)	36
5.7	Scenariusze bazowe: liczba replik (brak aktywnego HPA)	36

Spis tabel

2.1	Podsumowanie stosu technologicznego projektu — środowisko GCP/GKE .	12
3.1	Macierz badawcza 2×3 — identyfikatory scenariuszy testowych	13
3.2	Zasoby przydzielone serwisom	15
5.1	Wyniki scenariuszy bazowych (bez HPA)	32
5.2	Wyniki scenariuszy CPU-bound	32
5.3	Wyniki scenariuszy I/O-bound	32